Projektgruppe Praktische Mathematik
Technische Universität Berlin
Sekretariat MA 4-3

Einführung in die Programmiersprache

Python

Projektgruppe Praktische Mathematik *

16. April 2021

*Zuletzt bearbeitet: Christopher Woggon und Pablo Marhoff

Inhaltsverzeichnis

0 Vor dem ersten Termin **1**
0.1 Wie soll mit diesem Skript gearbeitet werden? . . . 1
0.2 Geschichte der Computer . . . 1
0.3 Aufbau des Computers . . . 4
0.4 Wie arbeitet der Computer? . . . 5

1 Vom Problem zum Programm **7**
1.1 Zum Begriff Algorithmus . . . 7
1.2 Teilschritte der Problemanalyse . . . 9
1.3 Prinzipieller Ablauf bei der Erstellung eines Programms . . . 11
1.4 Dokumentationstechniken (Struktogramme) . . . 11
1.4.1 Sequenz . . . 12
1.4.2 Verzweigung – Alternative Auswahl . . . 13
1.4.3 Iteration (Wiederholung) . . . 14

2 Kennenlernen der Entwicklungsumgebung **16**
2.1 Installation von Spyder . . . 17
2.1.1 Installation auf Linux (Ubuntu) . . . 17
2.1.2 Installation auf Windows . . . 18
2.2 Erste Einblicke in Spyder . . . 18
2.3 Python-Shell . . . 20
2.4 Editor . . . 22

3 Variablen und Kontrollstrukturen I **23**
3.1 Variablen . . . 23
3.1.1 Datentypen . . . 24
3.1.2 Rechenoperationen mit Variablen . . . 25
3.1.3 Eingabe von Daten . . . 25
3.2 Kontrollstruktur: if/else . . . 26
3.3 Programmierstil . . . 29
3.3.1 Einrückung . . . 29
3.3.2 Namenskonventionen für Variablen . . . 29

4 Datenstrukturen **30**
4.1 Liste (list) . . . 30
4.1.1 Adressierung und Manipulation einzelner Elemente . . . 31
4.1.2 Adressierung und Manipulation von Listenbereichen . . . 32
4.1.3 Strings – eine spezielle Form von Listen . . . 33
4.1.4 Verschachtelte Liste . . . 33
4.1.5 Exkurs: Call-by-Value vs. Call-by-Reference . . . 34
4.1.6 Kopieren von Listen . . . 35
4.2 Tupel . . . 36

4.3 Dictionary . . . 37
4.3.1 Erstellen von Dictionaries . . . 38
4.3.2 Hilfreiche Operatoren und Funktionen . . . 38
4.3.3 Verschachtelte Dictionaries . . . 41
4.3.4 Kopieren von Dictionaries . . . 41
4.3.5 Umwandlung in eine Liste und umgekehrt . . . 42

5 Kontrollstrukturen II **44**
5.1 while-Schleifen . . . 44
5.1.1 break – Vorzeitiges Verlassen einer Schleife . . . 46
5.1.2 continue – Vorzeitiger Abbruch einer Iteration . . . 47
5.1.3 Flussdiagramme break und continue . . . 47
5.2 for-Schleife . . . 48
5.2.1 Stolperfalle: Seiteneffekte von Änderungen an der Datenstruktur . . . 49
5.2.2 break und continue . . . 50
5.2.3 range()-Funktion . . . 50

6 Funktionen **51**
6.1 Was sind Funktionen? . . . 51
6.2 Wie verwendet man Funktionen? . . . 51
6.3 Naming Conventions . . . 53
6.4 Sichtbarkeitsbereich von Variablen . . . 54
6.5 Nützliche Funktionen . . . 56
6.6 Type Casting . . . 58
6.7 Rekursion . . . 58

7 Module, Dateioperationen, Plotting, Exceptions **59**
7.1 Module . . . 59
7.1.1 Eigenes Modul erstellen . . . 59
7.1.2 Beispiel math Modul . . . 60
7.2 Dateioperationen . . . 61
7.2.1 Dateien lesen . . . 61
7.2.2 Dateien schreiben . . . 62
7.2.3 Umgang mit CSV-Dateien . . . 63
7.3 Graphen plotten . . . 64
7.4 Ausnahmebehandlung . . . 68

8 Einblick in objektorientierte Programmierung **72**
8.1 Was ist OOP? . . . 72
8.2 Was sind die Vorteile von OOP? . . . 72
8.3 Klassen erstellen und Objekte generieren . . . 74
8.3.1 Zugriffsbeschränkung: Private Attribute . . . 76
8.4 Vererbung . . . 77
8.4.1 Überschreiben von Methoden . . . 78

0 Vor dem ersten Termin

Computer nehmen in unserem Leben immer mehr Raum ein, doch die meisten Benutzer*innen dieser Geräte betrachten ihren Computer mehr oder weniger ehrfürchtig wie eine Zauberkiste. Umfangreiche Softwarepakete vermitteln ihnen das Gefühl, dass das Gerät intelligent sei, sogar intelligenter als sie selber, denn: Kann es nicht lauter Dinge, die sie nicht können?

Der Blick auf die Tatsache, dass ein Computer nur das kann, was ihm einprogrammiert wurde, wird im Normalfall durch mehr oder weniger bunt schillernde Oberflächen versperrt.

Wenn man grundlegend begriffen hat, wie ein Computer funktioniert und Programme arbeiten, verblasst der magische Effekt der Rechner mit ihrer Software. Was übrig bleibt ist ein Werkzeug. Zu welchen Zwecken es eingesetzt wird, liegt in den Händen der Entwickler*innen.

Diese Einführung in die Programmiersprache Python soll Interessierten die Möglichkeit bieten, selber Programme zu schreiben. Es wird viel Wert auf eine strukturierte Programmierung gelegt und in diesem Zusammenhang die Verwendung von Struktogrammen verlangt.

0.1 Wie soll mit diesem Skript gearbeitet werden?

Es ist möglich, mit diesem Skript weitestgehend autodidaktisch zu arbeiten. Trotzdem kann und soll es kein vollständiges Python-Nachschlagewerk sein. Es ist deshalb empfehlenswert, für weiterführende Informationen entsprechende Bücher sowie das Internet zu Rate zu ziehen.

Die Kapitelnummern der im Kurs gestellten Aufgaben entsprechen den Kapitelnummern des Skripts. Zur Lösung der Aufgaben müssen die zuvor im Skript behandelten Themen praktisch angewandt werden.

0.2 Geschichte der Computer

Dieses Kapitel kann zu Kursbeginn übersprungen werden, und irgendwann in einer ruhigen Minute gelesen werden.

> Denn es ist eines ausgezeichneten Mannes nicht würdig, wertvolle Stunden wie ein Sklave im Keller der einfachen Rechnungen zu verbringen. Diese Aufgaben könnten ohne Besorgnis abgegeben werden, wenn wir Maschinen hätten.
>
> Gottfried Wilhelm Leibniz, 1646 - 1716.
> Er entwickelte u.a. das binäre Zahlensystem und konstruierte eine Rechenmaschine.

Die Entwicklung von Computern (Rechenmaschinen) ist stark an die Entwicklung der Mathematik und der physikalischen Naturwissenschaften gebunden. Die Entwicklung der Mathematik und der Naturwissenschaften sind wiederum stark an gesellschaftliche Entwicklungen gebunden.

Die Notwendigkeit Zahlen zu entwickeln, entstand mit dem Fortschreiten der Urbanisierung und der Staatengründung. Steuern sollten eingetrieben werden (zunächst noch in Naturalien), große Menschenmengen mussten versorgt werden, logistische Probleme gelöst und Handel getrieben werden. Nur in

Kulturen mit Stadt- und Staatsbildung hat sich eine Mathematik und ein größere Mengen beschreibendes Zahlensystem entwickelt.

Schon frühzeitig wurden Hilfsmittel zu der Bewältigung der anfallenden Rechenaufgaben gefunden. Sei es die Entwicklung des Abakus in Ostasien (ca. 1100 v.u.Z.) oder das Rechnen mit Steinen. Von Euklid (ca. 365-300 v.u.Z.) existiert ein Buch, das bereits zahlreiche Algorithmen, zur Vereinfachung von Rechenvorgängen enthält.

Ein entscheidender Schritt für die Weiterentwicklung der Mathematik war die Einführung der Null, die erst das uns gewohnte Rechnen im Dezimalsystem (und auch im binären Zahlenraum) ermöglicht. Die Null wurde ca. im 7. Jahrhundert unserer Zeitrechnung in Indien eingeführt, in Europa erst im 12. - 13. Jahrhundert, mit der Einführung des arabischen Zahlensystems.

Adam Ries (1492-1559), der in der Handelsstadt Nürnberg lebte, entwickelte Rechenbücher zur Formalisierung der dezimalen Rechenmethoden. Im siebzehnten Jahrhundert wurden von verschiedenen Leuten mechanische Rechenmaschinen entworfen (Schickardt, Blaise Pascal, Leibniz u.a.)[1].

Mit der Entwicklung des binären Zahlensystems und seiner Arithmetik legte Leibniz die Grundlagen für die späteren Computerentwicklungen.

Die nächsten entscheidenden Schritte für die Computerentwicklung finden erst im 19. Jahrhundert statt:

Jaquardt entwickelt zu Beginn des Jahrhunderts einen automatischen Webstuhl, der mit Lochkarten gesteuert wird. Charles Babbage – ein englischer Mathematiker, der sich mit Sterbetafeln für Versicherungen (Statistik) und astronomischen Tabellen (im Auftrag des Kriegsministeriums) beschäftigte – baut um 1830 die erste programmgesteuerte Rechenmaschine, die „Analytical engine“. Angeblich hat sie nie so richtig funktioniert. Seine Assistentin, Ada Lovelace, war die erste theoretische Informatikerin.

Um 1855 entwickelt George Boole die nach ihm benannte *boolesche Algebra*, die zum Auswerten von logischen Zusammenhängen notwendig ist und auch ein wesentlicher Bestandteil von Computerprogrammen ist.

Ende des Jahrhunderts bekam Hollerith auf einer Bahnfahrt in den USA die Idee, Daten auf Lochkarten zu speichern. Der Gedankenblitz traf ihn, als der Kontrolleur in seine Fahrkarte Löcher knipste, die an verschiedenen Stellen unterschiedliche Bedeutungen hatten (Fahrziel, Preisklasse oder ähnliches). Mit Hilfe der Lochkarten von Hollerith wurde um 1880 eine Volkszählung in den USA durchgeführt. Die Lochkarten wurden maschinell ausgewertet. Diese Erfindung führte zum Einsatz in der Buchhaltung von größeren Unternehmen und zur Betriebsüberwachung (Lagerhaltung, Arbeitszeiten, etc.). Hollerith war Mitbegründer einer Büromaschinenfirma, die sich 1924 in Industrial Business Machines Corporation (kurz: IBM) umbenannte.

Ein technikbegeisterter Bauingenieurstudent der TH Charlottenburg (heute TU Berlin) Namens Konrad Zuse[2], hatte von den immer gleichen Auslegungsberechnungen zur Statik die Nase voll und konstruierte die ersten Rechner in Relaistechnik in der Wohnung seiner Eltern. Den ersten funktionstüchtigen Rechenautomaten in Relaistechnik entwickelte er dann 1941. Zuse war damals bei der Firma Henschel mit der Berechnung der Statik von fliegenden Bomben beschäftigt. Seine Erfindung blieb von staatlichen

[1] Schickardt wollte astronomische Rechnungen automatisieren, Blaise Pascal war der Sohn eines Steuerpächters, den er mit seiner Maschine unterstützen wollte. Die wenigsten der in dieser Zeit entworfenen Maschinen funktionierten tatsächlich.

[2] Der sich sowohl mit dem „Kapital“ von Marx, als auch mit den Texten von Henry Ford auseinandersetzte.

Stellen weitestgehend unbeachtet, wahrscheinlich, weil sie sich nicht zur Verarbeitung großer Datenmengen eigneten, sondern eher für technische Berechnungen.

In dieser Zeit liefen in den USA und in England mehrere Entwicklungen parallel, um Rechenmaschinen zu entwickeln. Zum einen brauchte man ballistische Tabellen[3], die mittels Anfangswertlösern berechnet wurden. Dazu waren immer wieder die gleichen Rechenvorgänge nötig und es wurden viele Menschen gebraucht, diese von Hand zu berechnen. Zum Anderen wollte man den geheimen Code der Deutschen in der U-Boot-Kommunikation entschlüsseln (u.a. durch Alan Turing nach dem auch die Turing Maschine bzw. der Turing Test benannt wurde).

Mitte der 40er Jahre entwickelte John von Neumann ein Prinzip zum Aufbau von Computern (von-Neumann-Computer), der im wesentlichen noch die heutige Rechnergeneration beschreibt. 1946 wurde der erste mit elektronischen Röhren ausgestattete Computer in Betrieb genommen (ENIAC). Die Computer dieser Generation hatten einen immensen Platzbedarf und waren extrem temperaturanfällig.

Die ersten rechnerunabhängigen Programmiersprachen wurden ab Mitte der 50er Jahre entwickelt (FORTRAN, ALGOL). Wohingegen es davor noch darum ging, konkret zu berechnende Probleme an die eigene Maschine anzupassen, konnten Probleme nun durch die höheren[4] Programmiersprachen allgemeiner formuliert werden. 1955 wurde der erste Transistor-Rechner in Betrieb genommen, 1957 die ersten Magnetplattenspeicher.

In den sechziger Jahren setzten immer mehr Betriebe Computer ein. Sowohl für Banken und Versicherungen, als auch für ingenieurmäßige und wissenschaftliche Berechnungen wurden Rechner und Programme benötigt. Mit der Weiterentwicklung der Rechner wurden die Computer immer kleiner, leistungsfähiger und billiger. Damit geschriebene Programme nicht nur durch die Entwickler*innen verstanden und gewartet werden konnten, wurden Standards und DIN-Normen zur strukturierten Programmierung entwickelt. Ab 1970 wurden Halbleiterspeicher entwickelt, 1971 der erste Mikroprozessor (INTEL 4004) herausgebracht.

Ende der 70er Jahre wurde das Betriebssystem UNIX entwickelt, um ein netzwerkfähiges, leistungsstarkes Betriebssystem zu haben. Rechnernetze wurden zu der Zeit insbesondere im militärischen Bereich benötigt. Man beabsichtigte mehrere Computer so zu vernetzen, dass, auch bei Ausfall einer oder mehrerer Komponenten, im Notfall noch alle wichtigen Operationen ausgeführt werden konnten. Außerdem benötigte man ein System zum schnellen Datentransfer. Aus diesen Zeiten stammt auch das inzwischen weit bekannte „Internet“.

1977 wurden die ersten Personalcomputer (PC) herausgebracht. Anfang der Achtziger Jahre kamen jede Menge „Heimcomputer“ auf den Markt, die im wesentlichen nur zum Spielen von Computerspielen benutzt wurden. Die angebotene Software war mager, leistungsfähige Programme waren teuer. In den folgenden Jahren wurden die PCs jedoch immer billiger und leistungsstärker und fanden dadurch bei immer mehr Leuten Verbreitung. Außerhalb des mathematisch-technisch-naturwissenschaftlichen Bereiches wurden sie nun überwiegend zur Textverarbeitung genutzt.

1995 eroberte das Schlagwort „Multimedia“ die bundesdeutsche Gesellschaft. In der Zwischenzeit gab es immer weniger Haushalte, die keinen Computer besitzen, sei es um die Urlaubsfotos von CD-ROM

[3] Für jedes Geschoss musste man die Flugbahn bei verschiedenen Abschusswinkeln, Windrichtungen und -stärken kennen.
[4] Mit „höher“ ist eine stärkere Abstraktion der verwendeten Hardware gemeint.

auf den Bildschirm zu laden, im Internet zu surfen, der Hausverwaltung einen Brief zu schreiben oder den Anrufbeantworter und das Fax-Gerät zu ersetzen.

Im wissenschaftlichen Bereich sind vernetzte UNIX-Workstations heutzutage ein weitverbreitetes Arbeitsmittel. Ein modernes, gut organisiertes Netz kann eine Vielzahl von Softwarepaketen einer großen Zahl von Benutzer*innen zur Verfügung stellen. Es gibt hier, gerade für wissenschaftliche Zwecke, viele leistungsfähige, kostenfreie Programmpakete – sogenannte „Freeware“.

0.3 Aufbau des Computers

Computer sind Maschinen, die nur die Anweisungen ausführen, die ihnen mit Hilfe eines Programmes „beigebracht“ wurden, sie besitzen also keinerlei Intelligenz[5].

Im wesentlichen besteht ein Computer aus einer Zentraleinheit (Central Processing Unit: **CPU**) mit Prozessor und dem Hauptspeicher, sowie peripheren Geräten zur Ein- und Ausgabe (Input/Output oder kurz: I/O) und peripheren Speichereinheiten (z.B.: Festplatten, Diskettenlaufwerken). Diese Struktur geht noch auf den *von-Neumann-Computer* (siehe Abschnitt 0.2 Geschichte der Computer) zurück, dessen Aufbau im folgenden beschrieben wird[6]. Abbildung 1 veranschaulicht den Aufbau eines Computers.

Die **Zentraleinheit** bildet den Kern des Computers. Hier werden die Informationen verarbeitet. Ihre Komponenten sind:

- Der **Hauptspeicher**, in dem während des Programmablaufes alle für das Programm wichtigen Daten gespeichert werden, das sind sowohl Programmdaten und Verarbeitungsdaten, die andere Bestandteile bei Bedarf abrufen, als auch Zwischenergebnisse der Verarbeitung. In dem Hauptspeicher liegt auch der Speicherbereich für den Eingabe- und Ausgabepuffer. Alle Daten die ein- bzw ausgegeben werden, werden hier zwischengespeichert und weiterverarbeitet.
- Der **Prozessor**. Er besteht aus:
  - der **Steuereinheit**, die den Programmablauf steuert. Dazu liest sie die Befehle sequentiell, also hintereinander, aus dem Hauptspeicher und sorgt dafür, dass diese ausgeführt werden;
  - und der **Verarbeitungseinheit**, die die Operationen nach Vorgabe der Steuereinheit durchführt und die Ergebnisse an die vorgegebene Adresse im Hauptspeicher liefert.
- Steuereinheit und Verarbeitungseinheit sind in den moderneren Computern in einem Bauteil realisiert.

[5] Maschinelles Lernen und künstliche Intelligenz können als Ausnahmen bezeichnet werden, dies ist für die folgende Diskussion aber unwichtig.

[6] Prinzipiell ist der Aufbau moderner Rechner gleich geblieben. Die einzelnen Komponenten werden evtl. anders benannt oder zusammengefasst.

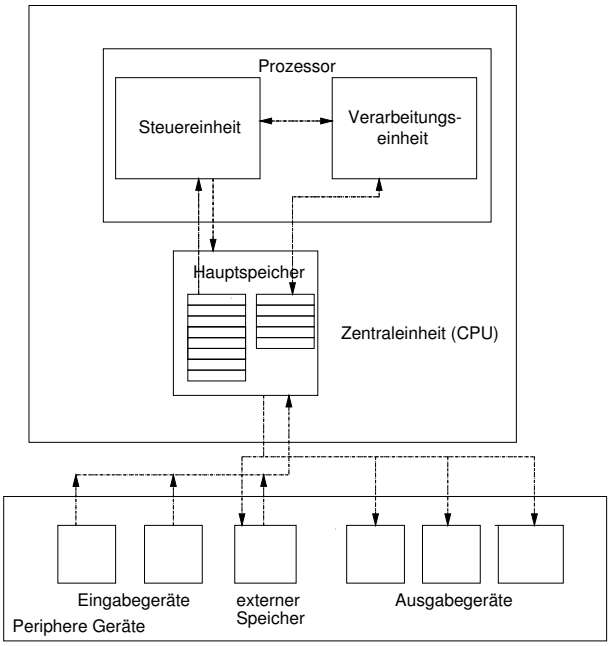


Abbildung 1: Prinzipieller Aufbau eines Computers

Über die **peripheren Geräte** findet die „Kommunikation“ des Computers mit der Umgebung statt. Man unterscheidet zwischen folgenden peripheren Geräten:

- Über die **Eingabegeräte** werden Programm- und Verarbeitungsdaten in den Hauptspeicher geladen (zum Beispiel: Tastatur, Maus).
- Auf die **Ausgabegeräte** werden die Resultate übertragen (zum Beispiel: Bildschirm, Drucker, Plotter).
- Die **externen Speicher** dienen zur Datensicherung. Sie können auch für Aus- und Eingaben verwendet werden (zum Beispiel: Festplatten, Diskettenlaufwerke, CD-ROM).

0.4 Wie arbeitet der Computer?

Ein Programm wird in den Hauptspeicher geladen und Schritt für Schritt abgearbeitet. Jede Einheit, die von dem Computer aus angesteuert werden kann, muss eine Adresse haben. Die Steuereinheit entnimmt dem Programm, von welchen Adressen Daten gelesen werden sollen, wie die Verarbeitungseinheit sie verarbeiten soll, und auf welche Adresse das Resultat geschrieben werden soll.

Jeder Speicherplatz und jedes Ein- und Ausgabegerät hat eine Adresse[7].

Man kann sich den Computer als eine Ansammlung von Schubkästen vorstellen, wobei auf jedem Schubkasten eine Adresse steht und man in diese Schubkästen Daten ablegen kann.

[7] Eigentlich „hängen“ Ein- und Ausgabegeräte an „Schnittstellen“ des Computers. Diese Schnittstellen haben wiederum Adressen.

Ein Programm, das vom Rechner ausgeführt werden kann, muss dem Computer in „seiner“ *Maschinensprache* vorliegen. Diese Maschinensprache ist von Computer zu Computer verschieden und beinhaltet grundlegende Operationen, die der Rechner ausführen kann (z.B. das Lesen und Schreiben von Daten aus einer angegebenen Adresse, etc.). Ein Programm ist letztendlich nichts anderes, als eine Liste von aneinander hängenden Anweisungen, die nacheinander abgearbeitet werden.

1 Vom Problem zum Programm

Der Ausgangspunkt jeder Programmentwicklung ist das Problem. In diesem Kapitel soll allgemein der Weg vom Problem zum Programm dargestellt und erläutert werden. Dieser Weg vom Problem bis hin zur Lösung durch ein Programm kann in einem ersten Schritt grob in drei Phasen unterteilt werden, die nacheinander durchlaufen werden:

- Problemanalyse
- Entwurf des Algorithmus
- Erstellung des Programms

Prinzipiell ist eine weitere Gliederung der Phasen in kleineren Abschnitten möglich bzw. notwendig, um den Ablauf hinreichend genau beschreiben zu können. Ein solches Vorgehen bezeichnet man auch als schrittweise Verfeinerung, auf die in diesem Kapitel noch näher eingegangen wird.

Zunächst aber werden allgemein der Begriff Algorithmus und – wegen der besonderen Bedeutung für eine gute Programmentwicklung – die Phase der Problemanalyse erläutert.

1.1 Zum Begriff Algorithmus

Das Wort Algorithmus ist ursprünglich abgeleitet vom Namen eines Persers (Abu Ja'far Mohammed ibn Musa **al Khowarizimi**, ca. 825 n. Chr.), der ein Lehrbuch über Mathematik geschrieben hatte. In neuen Lexika wird Algorithmus definiert als eine „spezielle Methode zur Lösung eines Problems".

Ein Algorithmus kann also – anders formuliert – verstanden werden als gemeinsamer Begriff für: Anleitung, Rezept, Vorschrift, Verhaltensmuster und Handlungsanweisung. Alle diese Begriffe stehen für eine Folge von Erklärungen „wie man etwas macht". Aus diesen Definitionen und Erklärungen wird deutlich, wie wichtig der Algorithmus für eine Problemlösung mittels Rechner ist. Denn der Rechner kann nur dann sinnvoll eingesetzt werden, wenn man ihm genaue d.h. für ihn „verständliche" Handlungsanweisungen gibt.

Ein ganz einfaches Beispiel für einen Algorithmus ist die bereits bekannte Handlungsanweisung zum Einschalten des Terminals bzw. zur LOGIN-Prozedur.

1. Terminal einschalten
2. LOGIN-Prozedur mit Angabe des USER-NAME beginnen
3. PASSWORD eingeben

Die wichtigsten Merkmale eines Algorithmus kann man schon an diesem einfachen Beispiel erkennen.

Der Algorithmus definiert einen *Prozess*. Eine wichtige Anforderung, die an den Prozess gestellt wird, ist, dass er hinreichend genau ist. Das bedeutet, dass jeder der o.g. Schritte, inklusive der beabsichtigten Reihenfolge, unmissverständlich sind, so dass der *Prozessor* (in diesem Fall eine Person an einem Computerterminal) diese versteht und sie ausführen kann.

Außerdem müssen sowohl die Beschreibung des Prozesses als auch der Prozess selbst *endlich* sein.

Selbstverständlich ergibt sich als weitere Anforderung an den Algorithmus, dass der Prozess, den er beschreibt, ausführbar ist. So ist zum Beispiel die Feststellung „Die LOGIN-Prozedur benötigt in jedem Fall die Angabe eines Passworts“ nicht ausführbar, und stellt folglich auch keinen Algorithmus dar. Dagegen wäre folgende Formulierung eine gültige Variante: „Bei einem Klick auf den Login-Button wird geprüft, ob das Passwort-Feld leer ist. Ist dies nicht der Fall, werden die Daten an den Server gesendet“.

Die Ausführung des Algorithmus erfolgt schrittweise und **sequentiell** (das heißt aufeinanderfolgend). Jeder der Teilschritte, aus denen sich der gesamte Algorithmus zusammensetzt, kann wieder als ein eigener (elementarer) Algorithmus betrachtet werden. Um umfangreiche Prozesse beschreiben zu können, bedarf es hingegen einen höheren Grad an Abstraktion. Die Wahl des Abstraktionsniveaus bedarf gründlicher Überlegung, da abgewogen werden muss, auf welcher Ebene (Abstraktionsebene bzw. Ebene elementarer Algorithmen) die Beschreibung des Algorithmus enden kann, um unnötige Details auszulassen und gleichzeitig hinreichend genau zu sein. Zum Beispiel reicht die Anweisung „Terminal einschalten“ in dem obigen Beispiel vollkommen aus. Weitere Erklärungen (Detaillierungen) wie zum Beispiel „Den Einschaltknopf kurz drücken. Warten bis das Hochfahren abgeschlossen ist. [...]“ wären überflüssig.

Bei der Erstellung eines Algorithmus sollte man einige grundsätzliche Kriterien im Auge behalten:

- Voraussetzungen:
  Unter welchen Bedingungen arbeitet der Algorithmus? Was sind zulässige Ereignisse, für die der Algorithmus funktioniert? Welche Einschränkungen liegen vor und was passiert bei falscher Eingabe?
- Termination:
  Ein endlicher Algorithmus führt nicht mit Sicherheit zu einem endlichen Prozess. Ist sichergestellt, dass der Prozess für alle Eingaben endet? Ist es unmöglich, den Prozess in einen nicht endenden Zyklus zu bringen, das heißt existieren geeignete Abfragen?
- Korrektheit:
  Realisiert der Algorithmus wirklich den gewünschten Prozess, bzw. wird die gestellte Aufgabe richtig gelöst? Funktioniert der Algorithmus entsprechend der Aufgabenstellung?
- Aufwand:
  Gibt es Möglichkeiten den Algorithmus zu verbessern? Könnte ein anderer Algorithmus zur Lösung desselben Problems mit weniger Aufwand ausgeführt werden? Für einen Algorithmus, der von einem Rechner ausgeführt wird, gehört zur Aufwandsabschätzung vor allem die Betrachtung des Speicherplatz- und des Rechenzeitbedarfs.

Zusammenfassend kann gesagt werden: Ein Algorithmus beschreibt genau und endlich in meist sequentieller Form einen Prozess. Die Beschreibung erfolgt in bestimmten Notationen und mit Hilfe abstrakter bzw. elementarer Algorithmen. Die verwendete Notation oder algorithmische Sprache wird durch die elementaren Algorithmen festgelegt. Aus dem bisher über Abstraktionsebenen Gesagten ist ersichtlich, dass man einen Algorithmus zur Lösung eines Problems mittels Rechner nicht gleich in der Programmiersprache erstellen sollte. Ein komplexes Problem ist oft nur durch die Einführung höherer Abstraktionsebenen durchschaubar und verständlich darzustellen. Durch verfeinernde, weitergehende Formulierung von Teilalgorithmen gelangt man schließlich von einer hohen Abstraktionsebene zur niedrigen. Dies sind in der Regel dann schon Elemente der Programmiersprache.

1.2 Teilschritte der Problemanalyse

Da bei der Umsetzung eines Problems in ein Programm die Phase der Analyse des Problems von besonderer Bedeutung ist, wird die Problemanalyse exemplarisch in einzelne wichtige Teilschritte zerlegt. Dies soll an einem Beispiel verdeutlicht werden:

Problemanalyse:

In der ausgiebigen Diskussion über die notwendigen Maßnahmen zur Sicherung des Energiebedarfs war und ist die Nutzung der Kernenergie äußerst umstritten. Gerade in den letzten Jahren rückte dabei das Problem der anfallenden Abfallprodukte in den Vordergrund.

Problemname:

In unserem Beispiel soll auf dieses Problem eingegangen werden. Es soll versucht werden, eine Lösungsstrategie (d.h. einen Lösungsalgorithmus) – natürlich nur beispielhaft und in sehr beschränktem Rahmen – zu entwickeln. Als erster Schritt muss das Problem namentlich definiert werden. Der Problemname, der in unserem Beispiel angebracht erscheint, lautet „Atommüllbeseitigung“. Das Problem ist damit benannt und schon grob umrissen.

Problemspezifikation:

Der zweite Schritt besteht darin, die Art des Problems zu analysieren und weiter zu spezifizieren. Es gilt also festzustellen, dass die anfallenden Abfallprodukte stark radioaktiv sind und für die Umwelt eine große Gefahr darstellen. Deshalb kann der „Atommüll“ nicht, wie andere Abfallprodukte, einfach auf die Müllhalde transportiert werden.

Abgrenzung:

Die Spezifikation, dass „Atommüll“ besonders gefährlich ist, reicht natürlich nicht aus, und es muss in einem nächsten Schritt versucht werden, das Problem weiter abzugrenzen. Die Abgrenzung besteht darin, dass unter „Atommüll“ in unserem Beispiel nur die abgenutzten Brennstäbe verstanden werden sollen. Unberücksichtigt bleiben andere zusätzlich entstehende Abfallprodukte (zum Beispiel radioaktives Kühlwasser). Des Weiteren soll die Menge des hochgiftigen Abfalls begrenzt sein.

Ziel:

Für eine Lösungsstrategie sind die bisherigen Schritte bei weitem noch nicht ausreichend, da überhaupt kein Lösungsziel angegeben wurde. Bezüglich der Atommüllbeseitigung existieren seitens der „Verantwortlichen“ eine Vielzahl von Zielen, welche die Strategie festlegen, als da wären: Wiederaufarbeitung und Entsorgung, Endlagerung in unterirdischen Mülldeponien usw. Eine weitere Möglichkeit, die bereits diskutiert wurde, ist die Errichtung von Endlagerstätten auf anderen Planeten (oder zum Beispiel auf dem Mond). Dieses Ziel soll in unserem Beispiel weiterverfolgt werden.

Mittel:

Die Ziele von Lösungsstrategien hängen natürlich in entscheidendem Maße von den Mitteln ab, die zur Verfügung stehen. Dazu gehören Kapital, Personal, Material, Maschinen, technisches Know-how, usw. Da die Mittel im Allgemeinen begrenzt sind, muss diesem Teilschritt besondere Bedeutung beigemessen werden.

Entscheidung:

Die vorangegangenen Teilschritte Abgrenzung, Ziel und Mittel haben das Problem soweit transparent gemacht, dass eine Entscheidung für einen bestimmten Lösungsweg möglich wird. Gehen wir in unserem Beispiel mal davon aus, dass genügend Kapital, Material und technisches Know-how zur Verfügung stehen. Unter diesen sehr günstigen Umständen kann eine Endlagerung der giftigen Abfallstoffe auf einem fremden Planeten ernsthaft in Betracht gezogen werden. Die Entscheidung versetzt uns nun in die Lage, eine grobe Handlungsanweisung bzw. einen Lösungsalgorithmus zu entwerfen.

Dieser erste grobe Algorithmus wird dann in weitere Teilschritte unterteilt, d.h. schrittweise verfeinert. Ein entscheidender Teilalgorithmus, der entwickelt werden muss, betrifft die Berechnungen der Raketenflugbahn von der Erde zum anderen Planeten. Diese Berechnungen werden sicher mit Hilfe eines Rechnerprogramms durchgeführt. Das verwendete Programm stellt dabei einen weiteren Unteralgorithmus dar.

Zum Schluss sei die Untergliederung der Problemanalyse in die einzelnen Teilschritte noch einmal graphisch mittels eines Struktogramms (dazu später mehr) in Abb. 2 dargestellt.

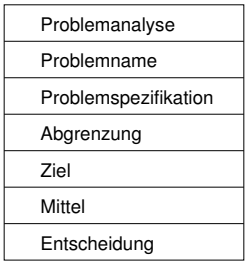


Abbildung 2: Struktogramm Problemanalyse

Grundsätzlich ist natürlich eine Problemanalyse denkbar, die eine ganz andere Handlungsanweisung (Algorithmus) ergibt. Der Algorithmus zur Endlagerung auf anderen Planeten ist überflüssig, wenn man bei der Analyse des Problems zu dem Schluss kommt, dass das Problem „Atommüllbeseitigung“ nur durch den Verzicht auf Kernenergie gelöst werden kann. Hieraus wird deutlich, welche entscheidende Bedeutung der Problemanalyse zukommt. Auch bei den kleinen Problemen innerhalb dieses Kurses sollte deshalb eine Problemanalyse durchgeführt werden.

1.3 Prinzipieller Ablauf bei der Erstellung eines Programms

Wie bei jeder Aufgabe, steht auch am Anfang jeder Programmentwicklung immer ein Problem. Der Weg vom Problem bis zur Lösung durch ein fertiges Programm ist ziemlich lang, und sollte deshalb in mehrere Phasen unterteilt werden.

Die einzelnen Phasen werden im folgenden dargestellt, um die Vorgehensweise zu veranschaulichen. Dabei werden zum Teil bereits Begriffe und Dokumentationstechniken verwendet, die erst später erläutert werden. Man lasse sich davon aber nicht irritieren, weil sie das grundsätzliche Verständnis nicht behindern.

Problemanalyse:

Das Problem (die Aufgabenstellung) wird zunächst benannt, untersucht und abgegrenzt, mit dem Ziel, die Eingangsdaten (Voraussetzungen) und die notwendigen bzw. erwünschten Ergebnisse möglichst genau zu beschreiben. Allgemein gehört dazu die Beschreibung des Ziels (was soll berechnet werden?) und die Sichtung der Mittel, die zur Verfügung stehen (zum Beispiel Näherungsverfahren, Computer usw.). Mit ihnen und der Abgrenzung (vgl. Seite 9) müssen die Entscheidungen getroffen werden, die zur Problemlösung führen.

Entwurf des Algorithmus:

Die Problemanalyse schafft die Grundlagen zur Entscheidung für das Lösungsverfahren. Beim Entwurf der Algorithmen wird dieses Verfahren näher beschrieben und dabei, gegebenenfalls mittels Verfeinerungen, weiterentwickelt. Dabei bedient man sich eines graphischen Hilfsmittels und zwar der Struktogramme (mehr dazu später). Die Entwicklung des Algorithmus bzw. seine Verfeinerungen sollten so weit gehen, dass ein Entwurf entsteht, der sich möglichst direkt in die Programmiersprache übertragen lässt.

Erstellung des Programms (Programmierung):

Der Algorithmus wird aus der sprachlichen und/oder graphischen Darstellung (Notation) in die Programmiersprache übertragen. Die Darstellung des Algorithmus ist im Allgemeinen so abstrakt, dass die Programmiersprache beliebig gewählt werden kann.

1.4 Dokumentationstechniken (Struktogramme)

Die Darstellung des Algorithmus erfordert eine Notation, auf die man sich einigen und dann festlegen muss, um auch anderen den Algorithmus transparent zu machen. Hierbei gibt es natürlich mehrere Möglichkeiten, mit deren Hilfe Algorithmen klar (anschaulich) und korrekt dargestellt werden können. Unter „korrekt“ wird an dieser Stelle verstanden, dass der Algorithmus genau das leistet, was beabsichtigt ist.

Ein Lernziel dieses Kurses ist die grafische Darstellung der Algorithmen mit Hilfe der Struktogramme, die nach ihren Erfindern auch **Nassi-Shneiderman-Diagramme** genannt werden. Diese Art der Darstellung soll in diesem Kapitel ausführlich erläutert werden. Alle Algorithmen haben die gleichen elementaren Grundstrukturen, die zu ihrer Darstellung benötigt werden.

Diese Grundstrukturen sind:

1. Sequenz: Aneinanderreihung von Teilalgorithmen
2. Verzweigung: Auswahl unter möglichen Teilalgorithmen
3. Iteration: Wiederholung von Teilalgorithmen

Diese Grundstrukturen reichen aus, um alle sequentiellen Algorithmen zu entwerfen.

Im weiteren Verlauf des Skriptes werden die Algorithmen nach diesen elementaren Grundstrukturen entwickelt. Das Ziel dabei ist, dass auch die mit Hilfe dieser Algorithmen entwickelten Programme nur noch diese Strukturen aufweisen. Programme, die auf diese Weise erstellt werden, sind in ihrem Aufbau und ihrer Logik viel leichter überschaubar. Das trägt sehr zur Transparenz der Programme bei. Diese Art der Programmentwicklung wird auch „strukturiertes Programmieren“ genannt. Im folgenden werden die Grundstrukturen der Algorithmen näher erläutert und ihre Darstellungen im Nassi-Shneiderman-Diagramm erklärt.

Die Struktogramme stellen die Teilalgorithmen oder Segmente in Form von Rechtecken dar. Bei der Sequenz mehrerer Segmente werden die entsprechenden Rechtecke übereinandergesetzt. Dabei ist der Ausgang des Einen der Eingang des Nächsten. Pfeile und Ähnliches entfallen, da die Abfolge der Segmente stets von oben nach unten geht.

1.4.1 Sequenz

Unter Sequenz versteht man eine Aneinanderreihung von Teilalgorithmen oder Segmenten, die als unteilbares Grundkonstrukt aufgefasst werden können. Diese Segmente sind Programmfragmente (entweder einzelne Anweisungen oder auch eine Folge mehrerer Anweisungen). Solche Segmente zeichnen sich dadurch aus, dass sie einen genau definierten Eingang und einen genau definierten Ausgang besitzen. In Abb. 3 ist ein Algorithmus zu sehen, der sich aus n Teil- oder Unteralgorithmen „Anweisung 1“, „Anweisung 2“,..., „Anweisung n“ zusammensetzt.

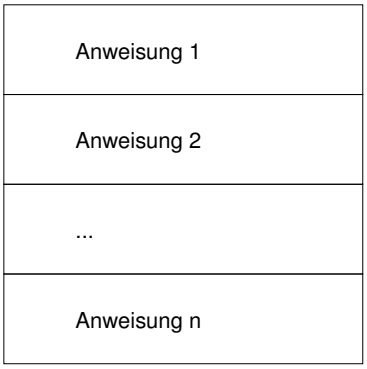


Abbildung 3: Struktogramme – Sequenz

Bei dieser Darstellung sind die Proportionen der Rechtecke ohne Bedeutung; sie werden dem Platzbedarf angepasst, der für den einzufügenden Text benötigt wird. Lediglich aneinander stoßende Ober- bzw. Unterkanten der Rechtecke müssen gleich lang sein.

1.4.2 Verzweigung – Alternative Auswahl

Die Verzweigung stellt eine Auswahl von Teilalgorithmen, unter der Abfrage einer Bedingung, dar. Die verschiedenen Varianten spielen bei der Entwicklung von Lösungsalgorithmen eine große Rolle. Diese „**Kontrollstrukturen**“ bilden nahezu den Kern eines jeden Programms.

Die Grundform ist die alternative Auswahl. Sie wählt in Abhängigkeit einer Bedingung, die wahr oder falsch sein kann, den entsprechend vorgesehenen Teilalgorithmus aus. Ist eine Bedingung erfüllt, so ist sie „wahr“ und nur der Teilalgorithmus „Anweisung 1“ soll ausgeführt werden. Im anderen Fall ist die Bedingung nicht erfüllt, also „falsch“, und der Teilalgorithmus „Anweisung 2“ soll ausgeführt werden. Die alternative Auswahl kann nun im Struktogramm, wie in Abb. 4 zu sehen, dargestellt werden.

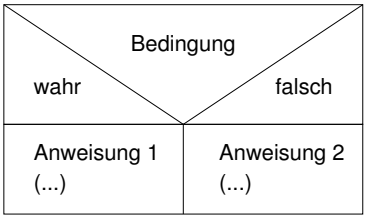


Abbildung 4: Struktogramme – Alternative Auswahl

Bedingte Auswahl

Eine weitere – spezielle – Form der Verzweigung ist die bedingte Auswahl. Diese entscheidet in Abhängigkeit von einer Bedingung, ob ein Teilalgorithmus bzw. weitere Programmsegmente ausgeführt werden sollen oder nicht. Ist die Bedingung erfüllt („wahr“) wird ein Teilalgorithmus ausgeführt. Formal ist die bedingte Auswahl ebenfalls eine Verzweigung, bei der lediglich bei nicht erfüllter („falsch“) Bedingung kein Teilalgorithmus ausgeführt wird. Das entsprechende Nassi-Shneiderman-Diagramm ist in Abb. 5 dargestellt.

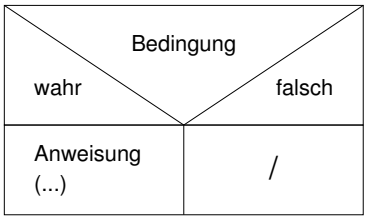


Abbildung 5: Struktogramme – Bedingte Auswahl

Eine bedingte Auswahl bietet sich besonders an, um sogenannte „verbotene“ Operationen (zum Beispiel Division durch Null) auszuschließen oder gegebenenfalls einen vorgegebenen Definitonsbereich einzuhalten.

Beispiel: Sollen in einem Programm ausschließlich reelle Quadratwurzeln berechnet werden, kann mit Hilfe der bedingten Auswahl das Berechnen der Wurzel nur dann durchgeführt werden, wenn die Zahl, deren Wurzel bestimmt werden soll, nicht negativ ist.

Mehrfachauswahl

Die Mehrfachauswahl ist eine Erweiterung der einfachen Alternative. Es werden mehrere Alternativen berücksichtigt. Jede der möglichen auswählbaren Aktionen wird durch einen Kennwert charakterisiert. Trifft keiner der Kennwerte zu, wird eine sonstige Handlung ausgeführt. Im Struktogramm wird die Mehrfachauswahl wie in Abb. 6 zu sehen dargestellt.

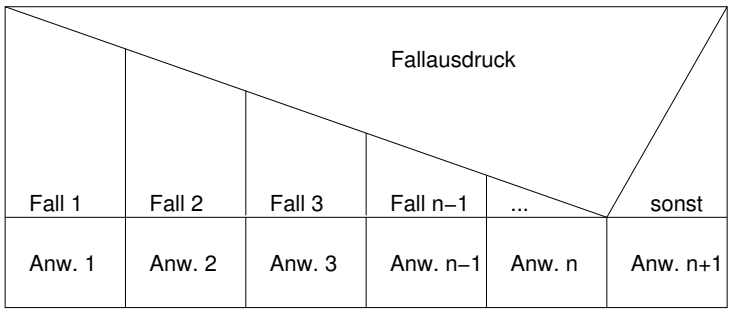


Abbildung 6: Struktogramme – Mehrfachauswahl

1.4.3 Iteration (Wiederholung)

Die Iteration ist die Wiederholung eines Teilalgorithmus in Abhängigkeit von einer Bedingung. Dabei sind verschiedene Formen möglich; eine Wiederholung mit Eintrittsbedingung (kopfgesteuert), eine Wiederholung mit Austrittsbedingung (fußgesteuert) und eine Wiederholung mit vorgegebener Zählvorschrift (Zählschleife).

Kopfgesteuerte Wiederholung

Am Anfang einer solchen Iteration steht eine Eintrittsbedingung, die überprüft wird. Ist die Bedingung erfüllt (wahr), wird der anschließende Teilalgorithmus so lange wiederholt, bis sie nicht mehr erfüllt (falsch) ist. Für den Fall, dass die Bedingung von Beginn an nicht erfüllt ist, gelangt das vorgesehene Programmsegment nicht zur Ausführung. Es findet also eine richtige Eingangskontrolle statt. Das Struktogramm hierfür ist in Abb. 7 zu sehen.

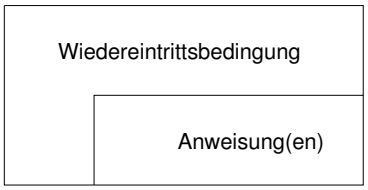


Abbildung 7: Struktogramme – Wiederholung kopfgesteuert

Fußgesteuerte Wiederholung

Bei dieser Form der Wiederholung steht der Teilalgorithmus (Anweisung) vor der (Austritts-) Bedingung. In diesem Fall wird, im Gegensatz zur Iteration mit Eintrittsbedingung, der Teilalgorithmus so lange wiederholt, bis die Bedingung zum „Austritt“ erfüllt ist. Zu beachten ist bei dieser Art der Wiederholung, dass der Teilalgorithmus mindestens einmal ausgeführt wird. Das Struktogramm hierfür ist in Abb. 8 zu sehen.

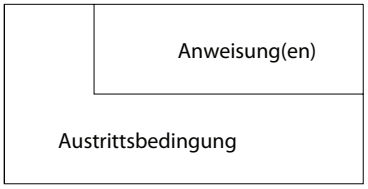


Abbildung 8: Struktogramme – Wiederholung fußgesteuert

Zählschleife (Wiederholung mit Zählvorschrift)



Abbildung 9: Struktogramme – Zählschleife

Bei der Wiederholung mit Zählvorschrift wird die Wiederholung eines Teilalgorithmus durch das Verändern einer Zählvariablen gesteuert. Einer solchen Zählvariablen wird dabei zunächst ein Anfangswert zugewiesen, der ständig pro Wiederholung um eine ebenfalls vorgegebene Schrittweite erhöht wird, bis eine festgelegte Bedingung nicht mehr erfüllt ist (z.B.: die Zählvariable ist nicht mehr innerhalb festgelegter Grenzen). Ist die definierte Bedingung nicht mehr erfüllt, erfolgt keine Ausführung des Teilalgorithmus mehr. Das Nassi-Shneiderman-Diagramm ist in Abb. 9 dargestellt.

Die drei Grundstrukturen Sequenz, Verzweigung und Iteration reichen – wie bereits zu Beginn des Kapitels erwähnt – im Prinzip aus, um für alle Probleme einen Lösungsalgorithmus zu formulieren.

Bei größeren Problemen, die komplexere Lösungsalgorithmen verlangen, muss dieser natürlich Schritt für Schritt in kleinere Teilalgorithmen unterteilt und verfeinert werden, bis letztlich hin zu den Grundstrukturen. Diese können dann mit relativ wenig Aufwand in die Programmiersprache übertragen werden.

Hier sei angemerkt, dass gute, schrittweise verfeinerte Algorithmen das Programmieren selbst sehr erleichtern. Außerdem hat eine solche Vorgehensweise den Vorteil, dass die Programme zwangsläufig gut aufgebaut sind, da ihr Aufbau gründlich durchdacht ist.

2 Kennenlernen der Entwicklungsumgebung

Um aus einer Idee ein lauffähiges Programm entwickeln zu können, müssen mehrere Schritte erfolgen. Zunächst muss der Code geschrieben werden. Hierfür kann ein normaler **Texteditor**[8] verwendet werden.

Es existieren einfache vorinstallierte Texteditoren, wie z.B. *Nano* (für Linux Ubuntu) oder *Editor* (Windows), und auch zusätzlich installierbare, wie beispielsweise *Atom* (Ursprung bei Ubuntu) oder auch *Notepad++* (Ursprung bei Windows). Letztere sind Open-Source-Programme, die im Gegensatz zu den vorinstallierten Varianten zusätzliche Funktionen besitzen, wie beispielsweise Syntax[9]-Hervorhebungen, Auto-Vervollständigungen oder auch automatische Einrückungen von Code-Zeilen. Das macht, wie Ihr im Laufe dieses Moduls feststellen werdet, sowohl das Programmieren aber insbesondere auch das Finden von Fehlern („Bugs“) wesentlich leichter.

In einem nächsten Schritt muss der Code in eine für den Computer verständliche Form gebracht werden. Vereinfacht gesagt, wird je nach Programmiersprache der Quelltext entweder zunächst **kompiliert** (in Maschinencode[10] übersetzt) oder direkt zur Laufzeit des Programms **interpretiert**. Typische Compiler-Sprachen sind Fortran95, C oder Pascal, die bekannteste Interpreter-Sprache ist Java.
Python wird grundsätzlich auch in die Gruppe der Interpreter-Sprachen eingeordnet. Das heißt, der Python-Interpreter (selbst ein vorkompiliertes Programm) führt zur Laufzeit Zeile für Zeile des geschriebenen Codes aus bzw. Code-Zeilen können direkt zur Laufzeit des Interpreters einzeln eingegeben und ausgeführt werden (s. Abschnitt 2.3). Allerdings können unter anderem Code-Dateien auch in kompilierter Form vorliegen, weshalb Python oft als eine „Mischform“ beschrieben wird.

In diesem Kurs werdet ihr für die Arbeit mit Python eine **IDE** verwenden. IDE steht für **I**ntegrated **D**evelopment **E**nvironment, im Deutschen auch **Entwicklungsumgebung** genannt, und ist in ihrem Umfang deutlich komplexer als ein Text-Editor. Genauer gesagt, handelt es sich um eine Sammlung von Computerprogrammen, die alle Schritte der Software-Entwicklung bündelt. Neben einem Editor mit Syntax-Hervorhebung, etc. sind bereits viele Standardbibliotheken sowie Werkzeuge zum Debuggen enthalten. Weitere hilfreiche Tools lassen sich leicht hinzufügen.

In eurem Studium und womöglich auch in eurem späteren Beruf werdet ihr mit verschiedenen IDEs arbeiten. Es gibt sie für nahezu alle Programmiersprachen und Plattformen. Für das Programmieren in Python haben sich verschiedene IDEs durchgesetzt. Neben denen, die für viele verschiedene Programmiersprachen verwendet werden können (z.B. *Eclipse*, *Sublime Text* oder *Visual Studio Code*), gibt es auch welche, die konkret für Python entwickelt wurden, wie beispielsweise *PyCharm*[11] oder auch *Spyder*.
Im Rahmen dieses Moduls verwenden wir Spyder, da der Einstieg relativ leicht ist und verschiedene anschauliche Werkzeuge zum Auffinden und Korrigieren von Fehlern (auch Debugging genannt) zur Verfügung stehen.

[8] Damit ist nicht Word gemeint! Dort werden in der .doc oder .docx-Datei neben dem eigentlichen Text eine Vielzahl an Meta-Informationen über Schriftart, Schriftgröße oder sonstige Formatierungen abgespeichert.
[9] Wie bei natürlichen Sprachen definiert *Syntax* in der Programmierung die Strukturierung von Wörtern und Zeichen zu erlaubten Konstruktionen
[10] Maschinencode besteht aus Befehlen, die direkt vom Prozessor verarbeitet werden können
[11] Nicht Open-Source! Neben einer umfangreicheren aber kostenpflichtigen Version existiert auch eine kostenlose Version.

In den folgenden Abschnitten werden zunächst der Installationsprozess und anschließend die ersten Schritte zur Nutzung von Spyder beschrieben. Anschließend werden die zwei Möglichkeiten zur Eingabe von Code vorgestellt (Python-Shell und Editor).

2.1 Installation von Spyder

An sich ist es möglich, Spyder direkt über deren Website zu installieren. Allerdings empfiehlt es sich, Spyder über das **Anaconda**-Paket herunterzuladen und zu installieren. Dieses Paket benötigt zwar etwas mehr freien Festplattenspeicher (rund 5 GB) für die Installation, jedoch beinhaltet es folgende zusätzliche Funktionen:

- es werden automatisch viele wichtige Standardbibliotheken wie NumPy, SciPy oder Matplotlib installiert, welche ansonsten manuell hinzugefügt werden müssten. Welche Rolle diese Bibliotheken spielen, wird in z.B. Abschnitt 7.1.2 oder auch in Abschnitt 7.3 erläutert.
- **Anaconda Navigator:** Eine grafische Oberfläche zum Installieren, Updaten und Starten der einzelnen Programme (u.a. Spyder), Aktualisieren von Paketen oder Erstellen von mehreren Conda-Umgebungen (auch Environments[12] genannt).

Wie ihr Anaconda auf Ubuntu oder Windows installiert, findet ihr in den folgenden Abschnitten. Weitere Infos zur Installation, etc. findet ihr auch online unter `https://docs.anaconda.com/anaconda/`

2.1.1 Installation auf Linux (Ubuntu)

1. Ladet die passende Python3-Version von der offiziellen Website herunter:
   `https://www.anaconda.com/download/`
2. Öffnet mit **Ctrl + Alt + T** ein neues Terminal.
3. Wechselt in das Verzeichnis, wo die Installationsdatei *Anaconda3-[...].sh* gespeichert wurde.
   Falls ihr bisher noch nie mit dem Terminal gearbeitet habt, findet ihr hier[13] die wichtigsten Befehle (vor allem `ls` und `cd`).
   Hinweis: Der Ort, wo Anaconda installiert wird, hat nichts mit dem Speicherort der Installationsdatei zu tun.
4. Folgt den Anweisungen (Lizenzvereinbarung akzeptieren und Installationsort bestätigen)
5. Die Frage „Do you wish the installer to initialize Anaconda3 by running conda init?“ empfehlen wir euch für Ubuntu mit **Ja** zu beantworten. Dadurch wird die bashrc angepasst und ihr könnt im Terminal – unabhängig davon in welchem Ordner ihr euch befindet – einen *conda*-Befehl eingeben.
6. Um die Installation zu aktivieren, gebt folgendes in das Terminal ein und bestätigt anschließend mit der Enter-Taste: source ~/.bashrc

Die am Ende vorgeschlagene Installation von PyCharm wird nicht benötigt.

[12] Wenn ihr an verschiedenen Projekten gleichzeitig arbeitet, die auf unterschiedlichen Python- oder Paket-Versionen basieren, können mehrere Environments erstellt werden. In diesem Kurs werdet ihr diese Funktion nicht benötigen.

[13] `https://wiki.ubuntuusers.de/Shell/Befehls%C3%BCbersicht/`

2.1.2 Installation auf Windows

1. Ladet die passende Python3-Version von der offiziellen Website herunter:
   `https://www.anaconda.com/download/`
2. Führt die .exe-Installatiosdatei aus und akzeptiert im nächsten Schritt die Lizenzvereinbarung.
3. Wählt einen Installationsort aus. (Achtung: Der Pfad darf keine Leerzeichen oder Sonderzeichen beinhalten!)
4. Wählt aus, ob ihr Anaconda zu eurer PATH-Umgebung hinzufügen wollt. Bei einer Installation in Windows empfehlen wir euch, dies **nicht** zu tun, da es zu kleineren Komplikationen mit anderer Software kommen kann. Stattdessen könnt ihr zur Verwendung vom *Anaconda Navigator* oder der *Anaconda Prompt* diese über das Start-Menü öffnen. Die Auswahl, dass Anaconda als Standardprogramm für Python verwendet wird, kann beibehalten werden.

Die am Ende vorgeschlagene Installation von PyCharm wird nicht benötigt.

2.2 Erste Einblicke in Spyder

Je nach Betriebssystem lässt sich Spyder wie folgt öffnen: Unter **Ubuntu** muss zunächst mittels **Ctrl + Alt + T** ein neues Terminal geöffnet werden. Hier kann entweder *anaconda-navigator* oder direkt *spyder* eingegeben werden, um das entsprechende Programm zu starten.
Unter **Windows** wurden während der Installation Shortcuts im Start-Menü erzeugt, mit denen sich der Anaconda-Navigator oder auch direkt Spyder öffnen lassen.

Beim ersten Starten von Spyder, wird euch vorgeschlagen, *Kite* zu installieren. Wir empfehlen euch, dies nicht zu tun, da euch auch von Spyder selbst gute Auto-Vervollständigungen angezeigt werden. Sofern ihr keine Pro-Version von Kite bezahlen wollt, bekommt ihr nur Werbung angezeigt, habt ansonsten aber keine wirklichen Vorteile.

Die Oberfläche von Spyder ist in Abb. 10 zu sehen. Auf die markierten Bereiche wird im Folgenden eingegangen.

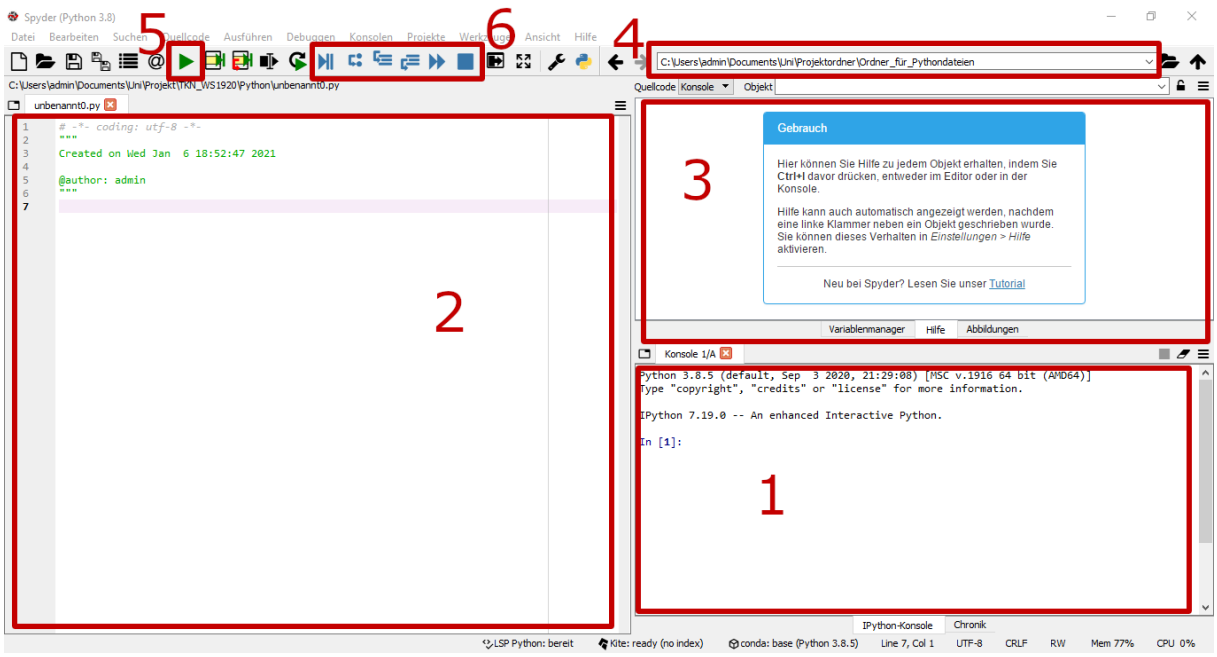


Abbildung 10: Oberfläche von Spyder

1 Die ***Python-Shell***, oder auch ***IPython-Konsole*** oder kurz ***Konsole***. Hier werden standardmäßig alle Programme ausgeführt, die in Spyder gestartet werden. Das bedeutet, dass z.B. Textausgaben, die ein gestartetes Programm generiert, hier ausgegeben werden. Unabhängig davon können auch einfach einzelne Variablen erstellt, bearbeitet oder ausgegeben werden. Genauere Infos findet ihr in Abschnitt 2.3. Außerdem sei noch zu erwähnen, dass über die Konsole auch auf höherer Ebene direkt mit Spyder interagiert werden kann (Installation von zusätzlichen Paketen, etc.).

2 Der ***Editor*** ist der Bereich, in dem ihr die meiste Zeit arbeiten werdet. Hier kann der Code für eure Programme erstellt und überarbeitet werden. Die Dateien, in denen euer Code abgespeichert wird, haben das Format *[Dateiname].py*. Wichtige Informationen für den Anfang findet ihr in Abschnitt 2.4.

3 in diesem Bereich könnt ihr euch verschiedene Informationen anzeigen lassen. Der Tab ***Variablenmanager*** zeigt euch den aktuellen Zustand aller Variablen an, die ihr erstellt habt (sowohl über die Konsole (1) als auch über den Editor (2)). Achtung: Ein neues Ausführen einer Datei (s. (5)) löscht alle alten Variablen! Dieser Bereich ist daher nicht zum Dauerhaften Speichern von Variablen geeignet.
Im Tab ***Abbildungen*** werden alle Abbildungen, die während eines Programmdurchlaufs erstellt werden, aufgelistet und temporär gespeichert. Wie ihr Abbildungen erstellt, werdet ihr in Abschnitt 7.3 kennenlernen.

4 Hier steht euer aktuelles ***Arbeitsverzeichnis***. Spyder wechselt beim Ausführen einer Datei automatisch in den Ordner, wo diese Datei liegt. Im späteren Verlauf des Kurses – wenn eure Programme ein bisschen komplexer werden – werdet ihr euren Programm-Code auf mehrere Dateien aufteilen müssen (s. Abschnitt 7.1.1). Spätestens dann werdet ihr genaueres über den PYTHONPATH lernen.

5 Über diese Schaltfläche bzw. mit der F5-Taste könnt ihr die aktuell im Editor (2) sichtbare ***Datei ausführen***.

6 Es wird oft vorkommen, dass ihr euren Programm-Code ausführen wollt (5), doch dabei in der Konsole (1) eine Fehlermeldung angezeigt wird und der Durchlauf abgebrochen wird. Lässt sich der „Bug“ nicht leicht finden, könnt ihr ***Datei debuggen*** (Strg + F5) auswählen. Mit ***Aktuelle Zeile ausführen*** (Strg + F10) könnt ihr anschließend jede Zeile eures Codes einzeln ausführen. Alternativ könnt ihr Haltepunkte (Breakpoints) setzen und mittels Strg + F12 die ***Ausführung bis zum nächsten Haltepunkt fortsetzen***. Im Variablenmanager (3) seht ihr jeweils den aktuellen Status eurer Variablen. Den Debugging-Modus könnt ihr verlassen, indem ihr auf ***Debugging stoppen*** (Strg + Umschalt + F12) klickt.

2.3 Python-Shell

In die Python-Shell (auch Konsole genannt) können einzelne Befehlszeilen eingegeben werden. Die Eingabeaufforderung (Prompt) `In [1]:` gibt dabei an, dass auf eine Eingabe (Input) gewartet wird. `[1]` entspricht einer laufenden Nummerierung und ist nicht weiter relevant.

Das typische Eingangsbeispiel fürs Programmieren ist ein Programm, dass lediglich die Textzeile „Hello world!“ ausgibt. In Python gibt es zum Ausgeben von Texten die Funktion `print()`. Ohne an dieser Stelle genauer darauf einzugehen, was eine *Funktion* ist, werdet ihr folgende Reaktion sehen können:

```
In [1]: print("Hello world!")          [ENTER-Taste drücken]
Hello world!
```

Statt *Hello world!* können selbstverständlich auch andere Texte eingegeben werden.

Außerdem könnt ihr – wie im folgenden Beispiel – in der Konsole einfache Rechenoperationen eingeben und Zwischenergebnisse in Variablen abspeichern. Neben den selbsterklärenden Operatoren `+`, `-`, `*` und `/` gibt es noch `%` (Modulo bzw. Rest) und ** (Exponentiation).

```
In [1]: 23+19
Out[1]: 42

In [2]: x=27**(1/3)          Hier wird x erzeugt und das Ergebnis der Gleichung abgespeichert

In [3]: x                    Durch Eingabe einer existierenden Variable wird der Wert abgerufen
Out[3]: 3.0

In [4]: y

Traceback (most recent call last):
  File "<ipython-input-18-9063a9f0e032>", line 1, in <module>
    y

NameError: name 'y' is not defined
```

Da `y` vorher nicht als Variable definiert wurde, erzeugt die Abfrage von `y` eine **Fehlermeldung** mit der Bezeichnung `NameError` und einer kurzen Beschreibung (inkl. Zeilenangabe!). Diese Fehlermeldungen

werden euch beim Programmieren sehr weiterhelfen, fehlerhafte Stellen zu finden. Kleiner Tipp: Die Bezeichnung des Fehlers sowie die kurze Beschreibung dahinter kann oftmals verwendet werden, um im Internet nach dem Fehler zu suchen.

Eventuell ist euch auch bereits aufgefallen, dass im Variablenmanager (Abb. 10 **(3)**) die Variable **x** aufgetaucht ist, sowie deren Datentyp und ihrem aktuellen Wert. Ihr könnt den Wert der Variable hier sogar „manipulieren“ (mit einem Doppelklick auf den Wert). Bei einer anschließenden Abfrage der Variable in der Konsole wird der neue Wert ausgegeben.[14]

Ebenso könnt ihr **logische Ausdrücke** eingeben. Logische Ausdrücke bzw. logische Operatoren werden automatisch ausgewertet und geben einen Wahrheitswert zurück. Dieser Wahrheitswert kann in Python `True` oder `False` sein (boolesche Logik). Es gibt zwei Arten von logischen Operatoren:

Vergleiche: `>` (*größer*), `<` (*kleiner*), `>=` (*größer oder gleich*), `<=` (*kleiner oder gleich*), `==` (*gleich*) und `!=` (*ungleich*).

Verknüpfungen: `and` bzw. `&` (*Konjuktion*), `or` bzw. `|` (*Disjunktion*) und `not` (*Negation*)

ACHTUNG: Der Vergleichsoperator == wird gerne mit der Zuweisung = verwechselt!

```
In [1]: 14 >= 14
Out[1]: True                Da 14 gleich 14 ist, entspricht dies einer wahren Aussage

In [2]: -5 == 5
Out[2]: False               -5 ist ungleich 5. Daher ist diese Aussage falsch

In [3]: a = 4               a wird der Wert 4 zugewiesen

In [4]: b = 13              b wird der Wert 13 zugewiesen

In [5]: x = False           x ist eine boolesche Variable mit dem Wert False

In [6]: ((a < b) or False) and not(x)
Out[6]: True                Aus den Werten von a, b und x ist die logische Verknüpfung wahr
```

Falls euch Aussagenlogik bisher im Studium nicht begegnet ist, findet ihr hier die Wahrheitstabellen der drei grundlegenden Verknüpfungen:

and oder **&**	True	False
True	True	False
False	False	False

or oder |	True	False
True	True	True
False	True	False

not	
True	False
False	True

Tabelle 1: Wahrheitstabellen der logischen Verknüpfungen *and*, *or* und *not*

Doch nun wieder zurück zur Nutzung von Spyder. Bei der Konsole muss zuletzt noch beachtet werden, dass alle Code-Eingaben und Variablen nur temporär gespeichert werden. Das heißt, wenn ihr Spyder schließt, werden alle Informationen gelöscht. Um hingegen richtige, immer wieder ausführbare Programme schreiben und diese abspeichern zu können, müsst ihr den Editor verwenden.

[14] Das sind an sich nur kleine „Spielereien“, aber es kann trotzdem manchmal praktisch sein.

2.4 Editor

Wenn ihr nicht nur einzelne Zeilen, sondern umfangreichere Programme schreiben wollt, müsst ihr den Editor verwenden (Abb. 10 **(2)**). Es können neue Programme begonnen aber selbstverständlich auch an gespeicherten weiter geschrieben werden. Bezüglich der Syntax unterscheidet sich der Editor nicht von der Konsole.

Beispielhafte Code-Ausschnitte, die ihr (sofern hilfreich) auch in eure Abgaben einarbeiten dürft, werden in diesem Skript immer wie folgt dargestellt:

```
1 # Das ist ein Kommentar
2 print("Hello world!") # Das hier auch
3 print("Dies ist nur ein kleines Testprogramm")
4 answer = 42
5 x = (13 + answer) ** 2
6 print("x = ", x)
```

Wenn ihr dieses Programm ausführt (Abb. 10 **(5)**), wird euch die generierte Ausgabe in der Konsole angezeigt. Probiert es aus!

Das Hashtag (bzw. Doppelkreuz) leitet einen Kommentar ein. Alles, was in dieser Zeile dahinter steht, wird vom Interpreter nicht beachtet. Achtet beim Programmieren immer darauf, ausreichend viele und – im Gegensatz zum Beispiel – sinnvolle Kommentare zu schreiben, denn Code ist oftmals nicht selbsterklärend!

Hinweis: Ihr könnt anstelle des ganzen Programms auch nur einzelne Zeilen ausführen lassen! Hierfür könnt ihr die F9-Taste drücken. Allerdings muss dabei die Abhängigkeit von einzelnen Zeilen beachtet werden. Codezeile 5 des obigen Beispiels kann nicht einzeln ausgeführt werden, ohne vorher die Variable answer (Zeile 4) definiert zu haben. Ansonsten folgt eine Fehlermeldung.

3 Variablen und Kontrollstrukturen I

In diesem Kapitel stellen wir euch zwei grundlegende Bestandteile der Programmiersprache Python vor – Variablen und das if/else Konstrukt. Ohne diese wäre die Abbildung von komplexen Abläufen in einem Programm nicht möglich.

3.1 Variablen

In der Programmierung ist es oft erforderlich, Zahlen, Texte und andere Daten zwischenzuspeichern und sie an verschiedenen Stellen des Programms wieder abzurufen. Ein einfaches Beispiel ist die Zwischenspeicherung eines Rechenergebnisses, um basierend darauf zu einem späteren Zeitpunkt Entscheidungen zu treffen und den Programmfluss zu verändern.

Dies ist mithilfe von **Variablen** möglich. Eine Variable kann man sich wie einen Behälter vorstellen, in dem Daten abgelegt werden können. Jede Variable hat einen eindeutigen Namen. Der Name ist eine Referenz, also ein Verweis auf die Daten, über den diese angesprochen werden können. Im Gegensatz zu anderen Programmiersprachen, z.B. wie bei C, muss in Python nicht explizit angegeben werden, welche Art von Daten in den Variablen gespeichert werden soll. Um eine Variable zu erstellen (Deklaration) und ihr einen Wert zuzuweisen (Initialisierung), kann ein beliebiger Variablenname, gefolgt von einem Gleichheitszeichen sowie dem Wert der Variable eingetippt werden. Im folgenden Beispiel wird **1337** als eine Zahl in einer Variable namens **num** gespeichert.

```
1 num = 1337
```

Es ist zu beachten, dass das Gleichheitszeichen nicht die Gleichheit der Variable *num* und der Zahl *1337* ausdrückt. In Python ist ein einfaches Gleichheitszeichen ein Zuweisungsoperator, welches das rechte Element dem linken Element zuweist. Um die Gleichheit der Elemente zu prüfen, wäre ein doppeltes Gleichheitszeichen (**==**) nötig.

Will man Zahlen mit Dezimalstellen speichern, ist dies auch problemlos möglich. Nachfolgend wird die Zahl *12,345* in einer Variable namens **`num_with_decimals`** gespeichert. Es ist zu beachten, dass das Dezimaltrennzeichen in Python ein Punkt und kein Komma ist.

```
1 num_with_decimals = 12.345
```

Ebenfalls lassen sich Texte in Variablen abspeichern. In der Programmierung redet man von Strings. Strings sind, auf tieferer Ebene, eine Aneinanderreihung (*string*) von Buchstaben. Damit klar definiert ist, dass es sich um einen String handelt, muss der Text zwischen zwei Anführungszeichen stehen. Nachfolgend wird der Text *„Python ist gar nicht so kompliziert.“* als String in der Variable **`text`** gespeichert.

```
1 text = "Python ist gar nicht so kompliziert."
```

Alternativ ist es auch möglich, Text mit einfachen Anführungszeichen zu definieren. Hier gibt es allerdings das Problem, dass dieses Zeichen zum Beispiel in englischen Texten oft als Apostroph verwendet

wird. In diesem Sonderfall könnte es zu Verwirrungen kommen, ob es sich um ein Apostroph oder das Ende des Strings handelt.

```
text = 'Python ist gar nicht so kompliziert.'
```

Es steht euch frei, auf welche Weise ihr Strings erstellt.

Möchte man in einem String ein Anführungszeichen verwenden, muss dieses Zeichen *maskiert* (engl. *escaped*) werden. Hierfür muss vor dem Zeichen ein Backslash (\) eingefügt werden. Gleiches gilt für eine Reihe anderer Sonderzeichen.

```
text = "Dieser String enthält Zeichen, welche \"maskiert\" wurden."
# Dieser String enthält Zeichen, welche "maskiert" wurden.
```

```
text = 'Dieser String enthält einfache Anführungszeichen, welche
↪  \'maskiert\' wurden.'
# Dieser String enthält einfache Anführungszeichen, welche 'maskiert'
↪  wurden.
```

3.1.1 Datentypen

Datentypen beschreiben verschiedene Arten von Daten. Wie bereits erläutert, muss für Variablen in Python kein expliziter Datentyp angegeben werden. Trotzdem wählt Python im Hintergrund einen passenden Datentyp aus. Nachfolgend werden einige grundlegende Datentypen erklärt. Es gibt zwar noch mehr, diese sind für den Anfang allerdings nicht relevant.

Tabelle 2: Grundlegende Datentypen

Datentyp	Art	Beschreibung	Beispiel
str	text	Der Datentyp str (String) wird für Texte und Buchstaben verwendet.	Hello World
int	numerisch	Der Datentyp int (Integer) wird für ganze Zahlen verwendet.	13
float	numerisch	Der Datentyp float (Floating-point number) wird für Zahlen mit Dezimalstellen verwendet.	12.5
bool	binär	Im Datentyp bool (Boolean) gibt es nur die Werte „True" und „False", also wahr und falsch.	True

Gesondert zu erwähnen ist **`None`**, ein sogenanntes *Literal*. Weist man einer Variable **`None`** zu, so will man damit ausdrücken, dass dieser Variable bisher kein Wert zugewiesen wurde. Dies ist hilfreich, wenn man eine Variable definieren will, aber erst später aussagekräftige Daten zuweisen will.

3.1.2 Rechenoperationen mit Variablen

Grundlegende Rechenoperationen habt ihr bereits kennengelernt. Es ist möglich, Variablen für Rechenoperationen verwenden. Im nachfolgenden Beispiel werden zwei Variablen mit zwei Werten initialisiert. Anschließend werden die Variablen addiert und das Ergebnis in einer dritten Variable gespeichert. Führt man das Skript aus, so wird nach der Initialisierung der beiden Variablen deren Summe, in diesem Fall *30.0*, in der Variablen **`total`** gespeichert und anschließend über die Konsole ausgegeben.

```
1 first_num = 12.5
2 second_num = 17.5
3 total = first_num + second_num
4 print(total)
```

Auch Strings lassen sich addieren. In diesem Fall hängt Python die Strings aneinander, sie werden konkateniert.

```
1 text_part_1 = "Python ist gar "
2 text_part_2 = "nicht so kompliziert."
3 full_text = text_part_1 + text_part_2
4 print(full_text)
```

Dieses Programm gibt *Python ist gar nicht so kompliziert.* über die Konsole aus.

Die Konkatenation von unterschiedlichen Datentypen (zum Beispiel Strings und Zahlen) ist nicht ohne weiteres möglich, sondern erfordert ein Umwandeln der Datentypen (Type Casting). Dies wird in Abschnitt 3.1.3 und ausführlicher in Abschnitt 6.6 beschrieben. Möchte man die Variablen unterschiedlichen Typs nicht konkatenieren sondern lediglich zusammen ausgeben, kann man dies mit einem print()-Befehl realisieren. Hierzu werden die Variablen durch Komma getrennt angegeben.

```
1 text = "Anzahl von Bundesländern:"
2 zahl = 16
3 print(text, zahl)
```

Dieser Codeabschnitt gibt folgenden Text über die Konsole aus: *Anzahl von Bundesländern: 16*. Es ist zu beachten, dass ein Leerzeichen zwischen den Variablen automatisch eingefügt wird. Dies ist bei der reinen Konkatenation von Strings nicht der Fall und muss manuell erledigt werden.

3.1.3 Eingabe von Daten

Oft ist es gewünscht, Variablenwerte erst zur Laufzeit festzulegen. Eine Möglichkeit dies zu erreichen ist die manuelle Eingabe von Werten über die Konsole. Einer Variablen kann ein Eingabewert wie folgt zugewiesen werden.

```
entry = input("Eingabe: ")
print(entry)
```

Sobald das **input()**-Statement erreicht wird, wird der String innerhalb der Klammern des **input()**-Statements über die Konsole ausgegeben[15] und die Ausführung so lange pausiert, bis eine manuelle Eingabe erfolgt. Es können nun Daten eingetippt und mit der Enter-Taste abgeschickt werden. Anschließend werden die eingegebenen Daten in der Variable **entry** gespeichert. Standardmäßig werden eingegebene Daten als Strings gehandhabt. Will man beispielsweise Zahlen eingeben, muss die Eingabe konvertiert werden, bevor sie in der Variable gespeichert wird. Hierfür muss der zu konvertierende Wert in Klammern gesetzt werden. Der gewünschte Datentyp wird vor die Klammern geschrieben. Einige Datentypen habt ihr bereits kennengelernt. Im nachfolgenden Beispiel werden zwei Zahlen vom Typ *int* eingegeben und deren Summe über die Konsole ausgegeben.

```
num_1 = int(input("Erste Zahl: "))
num_2 = int(input("Zweite Zahl: "))
total = num_1 + num_2
print("Summe:", total)
```

Es ist Vorsicht geboten, da es zu Problemen kommt, wenn der Eingegebene Wert nicht in den gewünschten Datentyp konvertiert werden kann. Gibt man zum Beispiel ein Wort anstelle einer Zahl ein, stürzt das Programm ab. Wie man solche Fehler abfängt, erfahrt ihr in Abschnitt 7.4.

3.2 Kontrollstruktur: if/else

Bisher wird der von uns geschriebene Code stur von Anfang bis Ende ausgeführt. Doch oft ist es sinnvoll, Teile des Codes nur unter bestimmten Bedingungen auszuführen. Hierzu kann eine if-Anweisung verwendet werden. Der enthaltene Code wird nur ausgeführt, falls (engl. *if*) die angegebene Bedingung erfüllt ist. Im unteren Beispiel ist die Bedingung ‘zahl > 5‘. Es wird also geprüft, ob der Wert der Variable ‘zahl‘ größer als ‘5‘ ist. Da die Variable ‘zahl‘ den Wert 10 enthält, ist die Bedingung erfüllt. Somit wird die Nachricht ‘Die Zahl ist größer als 5‘ über die Konsole ausgegeben.

```
zahl = 10
if zahl > 5:
    print("Die Zahl ist größer als 5")
```

Es ist wichtig zu verstehen, dass die Bedingung **zahl > 5** intern in den Datentyp **bool** umgewandelt wird. **zahl > 5** → **10 > 5** → **True**. Nur, wenn die Bedingung zu **True** aufgelöst wird, wird der entsprechende Codeblock ausgeführt. Es folgt ein Gegenbeispiel:

```
zahl = 3
if zahl > 5:
    print("Die Zahl ist größer als 5")
```

[15] Auf diese Weise kann der Nutzer darüber informiert werden, welche Art von Eingabe erwartet wird.

In diesem Fall würde die Bedingung wie folgt aufgelöst werden: **`zahl > 5`** → **`3 > 5`** → **`False`**. Somit würde der Codeblock nicht ausgeführt werden.

Euch ist vielleicht bereits aufgefallen, dass die print-Anweisung eingerückt wird. Der Codeblock, der nur unter der gegebenen Bedingung ausgeführt werden soll, muss um vier Leerzeichen oder einen Tabstopp eingerückt werden. Mehr dazu ist in Abschnitt 3.3 zu finden.

Nachfolgender, nicht mehr eingerückter Code wird ohne Rücksichtnahme auf die Bedingung ausgeführt. Das nachfolgende Skript gibt die Nachricht *Das Programm wurde beendet* über die Konsole aus, obwohl die Bedingung der if-Anweisung nicht erfüllt wurde.

```
1 zahl = 3
2 if zahl > 5:
3     print("Die Zahl ist größer als 5")
4 print("Das Programm wurde beendet")
```

Doch was, wenn man zusätzlich einen anderen Codeblock ausführen will, genau dann, wenn die Bedingung nicht erfüllt ist? Ein trivialer Ansatz wäre, eine weitere if-Anweisung zu verwenden, zum Beispiel so:

```
1 zahl = 10
2 if zahl > 5:
3     print("Die Zahl ist größer als 5")
4 if zahl <= 5:
5     print("Die Zahl ist kleiner oder gleich 5")
6 print("Das Programm wurde beendet")
```

Das Problem ist, dass sich die Bedingung zwischen den if-Anweisungen ändern kann. Zum Beispiel könnte sich der Inhalt der Variable ‘zahl‘ im Codeblock der ersten if-Anweisung verändern.

```
1 zahl = 10
2 if zahl > 5:
3     print("Die Zahl ist größer als 5")
4     zahl = 3
5 if zahl <= 5:
6     print("Die Zahl ist kleiner oder gleich 5")
7 print("Das Programm wurde beendet")
```

In diesem Beispiel würden beide Codeblöcke der if-Anweisungen ausgeführt werden. Will man sich aber nur auf den initialen Wert der Variable *zahl* beziehen, sollte man die if-Anweisung um einen else-Block erweitern. Else ist Englisch für sonst bzw. anderenfalls.

```
zahl = 10
if zahl > 5:
    print("Die Zahl ist größer als 5")
else:
    print("Die Zahl ist kleiner oder gleich 5")
print("Das Programm wurde beendet")
```

Der else-Block wird nur ausgeführt, wenn die if-Bedingung zu Beginn der Kontrollstruktur nicht erfüllt ist. Somit ist sichergestellt, dass immer genau einer der beiden Codeblöcke ausgeführt wird.

Es können auch mehr als zwei Codeblöcke angegeben werden. Hierzu wird eine beliebige Anzahl von elif-Anweisungen zwischen die if- und else-Anweisungen eingefügt. Die Anweisung **elif** entspricht der Kombination aus **el**se und **if**.

```
zahl = 55
if zahl > 60:
    print("Die Zahl ist größer als 60")
elif zahl  > 40:
    print("Die Zahl ist größer als 40")
elif zahl > 20:
    print("Die Zahl ist größer als 20")
else:
    print("Die Zahl ist kleiner oder gleich 20")
print("Das Programm wurde beendet")
```

Auch hier wird maximal ein Codeblock ausgeführt. Obwohl die Bedingungen **`zahl > 40`** und **`zahl > 20`** beide erfüllt sind, wird nur der Codeblock der weiter oben stehenden Bedingung ausgeführt. Man könnte das gegebene Beispiel wie folgt umschreiben:

```
zahl = 55
if zahl > 60:
    print("Die Zahl ist größer als 60")
else:
    if zahl  > 40:
        print("Die Zahl ist größer als 40")
    else:
        if zahl > 20:
            print("Die Zahl ist größer als 20")
        else:
            print("Die Zahl ist kleiner oder gleich 20")
print("Das Programm wurde beendet")
```

3.3 Programmierstil

Nachfolgend werden einige Best Practices zum Thema Programming Style vorgestellt.

3.3.1 Einrückung

Da Python im Gegensatz zu vielen anderen Programmiersprachen nicht mit Klammern arbeitet, um Codeblöcke zu definieren, ist es äußerst wichtig, auf korrekte Einrückung zu achten. Bisher habt ihr Einrückungen schon im Kontext von if-Anweisungen kennengelernt. Hier wird der bedingt auszuführende Codeblock eingerückt. Rückt man eine Zeile zu viel oder zu wenig ein, funktioniert das Programm entweder gar nicht, oder anders als gewünscht. Man kann eine Zeile entweder über einen Tabstopp oder vier Leerzeichen um eine Ebene einrücken. Während die Geschmäcker auseinandergehen, wird offiziell dazu geraten, Code mit vier Leerzeichen einzurücken. Zum Beispiel bei verschachtelten if-Anweisungen ist es notwendig, teile des Codes mehrere Ebenen tief einzurücken.

In nachfolgenden Bild sind alle Codeblöcke mit einem schwarzen Kasten markiert. Jeder Kasten muss exakt eine Ebene tiefer eingerückt werden, als der umliegende Kasten.



3.3.2 Namenskonventionen für Variablen

In Python werden Variablen grundsätzlich kleingeschrieben. Besteht der Name der Variable aus mehreren Teilen, werden diese durch einen Unterstrich getrennt. Soll eine Variable also das aktuelle Jahr speichern, wäre der Name **`aktuelles_jahr`** anderen Möglichkeiten wie **`aktuellesjahr`**, **`aktuellesJahr`** oder **`AktuellesJahr`** vorzuziehen.

Grundsätzlich sind aussagekräftige Variablennamen anderen Möglichkeiten wie **`x`** oder **`list`** vorzuziehen.

4 Datenstrukturen

Bisher habt ihr hauptsächlich Datentypen kennengelernt, mit denen sich einzelne Werte abspeichern lassen (Integer, Float, Boolean). Müssen jedoch größere Datenmengen abgespeichert oder verarbeitet werden, fällt schnell auf, dass das Abspeichern in einzelnen Variablen sehr umständlich wird.

```
# Messwerte eines Sensors
val_0 = 22.3
val_1 = 22.4
val_2 = 22.6
val_3 = 22.9
val_4 = 23.5
val_5 = 24.8
# ...
```

Um die einzelnen Wert zu gruppieren, existieren in Python eine Vielzahl an zusammengesetzten Datenstrukturen. Die vielseitigste unter ihnen ist die **Liste** (list).

4.1 Liste (list)

Eine Liste entspricht einer geordneten Aneinanderreihung von einzelnen Objekten (items), die durch Kommata getrennt und von eckigen Klammern umschlossen werden. Mit „geordnet“ ist gemeint, dass die Position der Objekte statisch ist und genutzt werden kann, um einzelne Objekte zu adressieren. Ein wichtiger Punkt, den ihr euch dabei merken müsst: **Die Adressierung beginnt in Python immer mit der Null!**

Ihr könnt euch eine Liste als eine Reihe von Boxen vorstellen, die außen mit ihrer Position markiert sind und Platz für genau ein Objekt (z.B. Integer-Zahl oder String) haben. In Python können Listen nicht nur mit je einem Datentyp gefüllt werden, sondern auch (im Gegensatz zu vielen anderen Sprachen) **mehrere Datentypen als Einträge** besitzen. Dies wird in der Praxis allerdings eher selten angewendet, da verschiedene Datentypen unterschiedliche Operationen (Addition von Zahlen, Konkatenation von Strings) erlauben.
Achtung: Ein häufiger (und oftmals schwer auffindbarer) Fehler ist das Vertauschen von ***Dezimaltrennzeichen (Punkt)*** und ***Trennzeichen der einzelnen Werte (Komma)***!

```
#          0     1     2     3     4     5
vals = [22.3, 22.4, 22.6, 22.9, 23.5, 24.8]

#                int    String    float
random_stuff = [429, 'Ubuntu', 123.4]
```

Anstatt sechs verschiedene Variablen zu benötigen, werden alle sechs Werte unter der Variable **`vals`** als Liste abgespeichert.

Auf Listen-Objekten können außerdem eine Reihe von Funktionen ausgeführt werden, mit denen ihr Informationen zu einer konkreten Liste erhalten bzw. diese manipulieren könnt. Zum Beispiel kann mittels **`len()`** die Länge einer Liste abgefragt werden.

```
vals = [22.3, 22.4, 22.6, 22.9, 23.5, 24.8]

print(len(vals)) # 6
```

Weitere wichtige Funktionen werden zum Teil in den folgenden Abschnitten erwähnt. Eine umfangreichere Erklärung des Begriffs und des Prinzips von Funktionen erfolgt in Abschnitt 6.

4.1.1 Adressierung und Manipulation einzelner Elemente

Einzelne Werte oder auch Listen als Ganzes können jederzeit ausgelesen oder überschrieben werden. Die einzelnen Elemente werden dabei durch ihren Index wie im folgenden Beispiel adressiert.

```
vals = [22.3, 18.2, 22.6, 22.9, 23.5, 24.8]

print(vals)      # [22.3, 18.2, 22.6, 22.9, 23.5, 24.8]
print(vals[0])   # 22.3

vals[3] = 0.0
vals[1] = 22.4
print(vals)      # [22.3, 22.4, 22.6, 0.0, 23.5, 24.8]
```

In diesem Beispiel seht ihr, dass durch Abrufen eines einzelnen Listeneintrags keine Liste, sondern das an dieser Stelle enthaltene Objekt zurückgegeben wird.

In Python kann die Adressierung auch „rückwärts“ erfolgen! Der ***letzte Wert*** einer Liste kann mit **`[-1]`** angesprochen werden, der vorletzte mit **`[-2]`** usw.

```
vals = [22.3, 22.4, 22.6, 22.9, 23.5, 24.8]

print(vals[-2]) # 23.5

vals[-3] = 0.0
vals[-1] = 27.0
print(vals)      # [22.3, 22.4, 22.6, 0.0, 23.5, 27.0]
```

Im Gegensatz zur Programmiersprache C kann eine Liste in Python auf einfache Weise mittels **`append()`** um einzelne Elemente ergänzt werden. Diese werden am Ende der Liste eingefügt. Das Gegenstück zu `append()` ist **`pop()`**, womit ein einzelnes Element abgerufen werden kann, welches gleichzeitig aus der Liste entfernt wird.

```
1  even = [0, 2, 4]
2
3  even.append(6)     # [0, 2, 4, 6]
4  even.append(8)     # [0, 2, 4, 6, 8]
5
6  num = even.pop()   # [0, 2, 4, 6]
7  print(num)         # 8
8
9  # optionaler Parameter bei pop() gibt Index des Elements an
10 even.pop(1)        # [0, 4, 6]
```

4.1.2 Adressierung und Manipulation von Listenbereichen

Mit einem Doppelpunkt können **Listenbereiche** (auch Substrings genannt) erzeugt werden. Dieses Verfahren nennt sich „*Slicing*“ und das Ergebnis wird auch in Form einer Liste zurückgegeben.
Start- und Endindex des Listenbereichs können, aber müssen nicht angegeben werden. Werden sie weggelassen, werden alle folgenden Werte bis zum entsprechenden Ende der Liste zurückgegeben. Der Startwert ist immer in dem erzeugten Substring enthalten, der Endwert ist selber nicht mehr enthalten.

```
1  vals = [13, 17, 32, 35, 42, 44]
2
3  print(vals[1:4])   # [17, 32, 35]
4  print(vals[2:3])   # [32]
5  print(vals[2:])    # [32, 35, 42, 44]
6  print(vals[:2])    # [13, 17]
7  print(vals[3:666] # [35, 42, 44]
8
9  txt = ['apfel', 'Banane', 'KIRSCHE']
10 print(txt)         # gibt die Liste aus
11 print(txt[:])      # erzeugt eine Kopie der Liste und gibt diese aus
```

Wie ihr in Zeile 7 sehen könnt, führt die Angabe eines Bereichs, der über die Liste hinaus geht, nicht zu einer Fehlermeldung. Es werden lediglich die Werte zurückgegeben, die tatsächlich existieren.
Achtung: Durch die eckigen Klammern und den Doppelpunkt in Zeile 11 wird nicht die Liste selber zurückgegeben (wie mit `print(txt)`), sondern zunächst eine Kopie erstellt, die anschließend ausgegeben wird (mehr zum Kopieren von Listen in Abschnitt 4.1.6). Dies führt zwar in diesem Beispiel zum gleichen Ergebnis, sollte aber auch bezüglich der Laufzeit beachtet werden!

Sollen mit einem Mal ***mehrere Einträge*** zu einer Liste ***hinzugefügt*** werden, kann nicht **`append()`** verwendet werden! Für diesen Zweck gibt es stattdessen **`extend()`**.

```
odd = [1, 3, 5]

# odd.append([7, 9]) würde zu [1, 3, 5, [7, 9]] führen
odd.extend([7, 9])  # [1, 3, 5, 7, 9]
```

4.1.3 Strings – eine spezielle Form von Listen

Wenn ihr euch an das letztes Kapitel erinnert, wisst ihr, dass ein „String“ eine Aneinanderreihung von einzelnen Zeichen (Chars) ist. Diese könnt ihr, wie bei anderen Listen, einzeln oder als Substring abrufen.

```
txt = 'Tomatensalat'
print(txt[2:6]) # mate
```

Strings sind allerdings im Gegensatz zu anderen Listen im Nachhinein **nicht manipulierbar**, können also nicht verändert werden! Der Versuch einzelne Zeichen zu überschreiben, würde zu einer Fehlermeldung führen. Stattdessen könnt ihr (Sub-)Strings in eine neue Variable kopieren und dabei den Inhalt anpassen:

```
txt = 'Tomatensalat'

# txt[0:6] = 'Paprika'       FALSCH! Führt zu einem TypeError
txt = 'Paprika' + txt[-5:] # RICHTIG
print(txt)                 # 'Paprikasalat'
```

Auch wenn die neue Variable *txt* den gleichen Namen hat, wird im Hintergrund eine neue Variable erstellt, in der nun „Paprikasalat“ abgespeichert ist.

Es gibt in Python also nachträglich veränderbare (**mutable**) und nicht nachträglich veränderbare (**immutable**) Datenstrukturen. Neben Strings sind z.B. auch Tupel (s. Abschnitt 4.2) immutable Objekte.

4.1.4 Verschachtelte Liste

Innerhalb einer Liste können sich weitere Listen befinden. Die Verschachtelungstiefe ist hierbei nicht beschränkt. Wie verschachtelte Listen gebildet bzw. Elemente adressiert werden, die sich in einer tieferen Ebene befinden, lässt sich am besten an einem Beispiel zeigen:

```
# das 0. Element der Liste ist das einzelne Zeichen a, das 1. Element
↪ ist eine Liste mit drei Elementen
deep_list = ['a', [['b', 'c'], [['d']], 'e']]

print(deep_list[0])        # a
print(deep_list[1])        # [['b', 'c'], [['d']], 'e']
print(deep_list[1][0][1]) # c
print(deep_list[1][1])     # [['d']]
```

4.1.5 Exkurs: Call-by-Value vs. Call-by-Reference

Beim Kopieren einer Liste aber auch anderer Datenstrukturen gibt es verschiedene Möglichkeiten. Dabei muss ein entscheidender Punkt beachtet werden: Soll tatsächlich eine Kopie der Daten erzeugt werden oder nur eine neue Variable, die auf die gleichen Daten zeigt? Zu Beginn kann diese Unterscheidung ein bisschen verwirrend sein, daher hier ein kleines Beispiel:

*Anton hat sich das Skript zum Python-Kurs ausgedruckt. Barbara hätte auch gerne Zugriff darauf. Da es das Skript (in diesem Beispiel) nicht online gibt, fragt Barbara Anton. Nun gibt es zwei Möglichkeiten: Entweder **kopiert sie sich die ganzen Seiten vom Skript** und kann darin frei markieren oder wichtige Dinge für sich ergänzen. Auch Anton kann in seiner Version markieren und ergänzen. Beide machen das nur für sich und alle Markierungen/Notizen/... haben logischerweise keine Auswirkungen auf den Ausdruck der jeweils anderen Person.*
*Die andere Möglichkeit wäre, dass sich Anton und Barbara **den einen Ausdruck teilen**. Es werden keine Skript-Seiten kopiert. Stattdessen gibt Anton den Ort, wo sich das Skript befindet, an Barbara weiter („die Adresse wird kopiert"). Sie haben nun beide Zugriff auf das gleiche Heft. Alle Änderungen, die die eine Person daran vornimmt, ist auch für die andere Person sichtbar.*

Die erste Form, bei der die Daten tatsächlich kopiert werden, wird in der Informatik **Call-by-Value** genannt. Wird nur eine weitere Referenz erzeugt, die auf die „Originaldaten" verweist, spricht man von **Call-by-Reference**.

Beide Formen des „Kopierens" haben ihr Vorteile. Handelt es sich zum Beispiel um große Datenmengen, die nur ausgelesen werden sollen, wäre es viel zu aufwendig die Daten zu kopieren. Eine Weitergabe der Speicheradresse (auch Pointer genannt, da die Adresse auf die Daten *zeigt*) zu den Daten ist effizienter. Allerdings muss beachtet werden, dass sich Änderungen an den Daten bei einer Übergabe als Referenz auch auf die Ursprünglichen Werte auswirken.

4.1.6 Kopieren von Listen

Nun zur konkreten Implemetierung in Python. Es existieren ***drei Varianten*** zum Kopieren von Listen, die je nach Anwendungsfall besonders geeignet sind, aber auch zu unerwarteten Ergebnissen führen können. Die Varianten unterscheiden sich darin, dass sie nur Referenzen zu den Listenobjekten erzeugen (Call-by-Reference) oder aber die Daten tatsächlich in eine neue Liste kopieren (Call-by-Value).

1. '='-Zeichen

Der Inhalt der Liste wird **nicht** kopiert, nur die Adresse. Werden die in der Liste gespeicherten Daten verändert, wirkt sich dies auf alle Kopien aus. Man kann an dieser Stelle also von „Call-by-Reference" sprechen.

```
original = ['a', 'b', 'c']
first_copy = original

original.append('d')          # Änderung gilt für Original und Kopie
print(first_copy)             # ['a', 'b', 'c', 'd']

second_copy = first_copy
second_copy.clear()           # löscht alle Elemente
print(original, first_copy) # [] []
```

2. `copy()` oder `[:]` *(für immutable Objekte)*

Enthält eine Liste **immutable Objekte**, also `int`, `float`, `bool`, `str` oder `tuple` (s. Abschnitt 4.2), so werden mittels `copy()` bzw. `[:]` diese **Elemente der Liste kopiert** und unter der neuen Liste abgespeichert. Änderungen an Kopie oder Original wirken sich nicht auf die jeweils andere Liste aus.

```
original = ['a', 'b', 'c']
first_copy = original[:]
original.append('d')

second_copy = first_copy.copy()
second_copy.append('e')

print(original)                  # ['a', 'b', 'c', 'd']
print(first_copy)                # ['a', 'b', 'c']
print(second_copy)               # ['a', 'b', 'c', 'e']
```

Aber Achtung: Sind mutable Elemente in der Liste enthalten – also Datenstrukturen wie z.B. verschachtelte Listen – so wird deren Inhalt nicht kopiert! Stattdessen werden nur Referenzen erzeugt. Das heißt, von Objekten der inneren Liste (damit ist die *Liste in der Liste* gemeint) werden nur Referenzen kopiert, wie an diesem Beispiel erkennbar:

```
1  original = [['a', 'b'], ['c']]
2  first_copy = original[:]
3
4  original[1].append('x')
5  print(original)         # [['a', 'b'], ['c', 'x']] wie erwartet
6  print(first_copy)       # [['a', 'b'], ['c', 'x']] 'x' auch enthalten!
7
8  original.clear()
9  print(original)         # []
10 print(first_copy)       # [['a', 'b'], ['c', 'x']]
```

Das Löschen aller Elemente in Zeile 8 wirkt sich wie erwartet nur auf die Liste **`original`** aus. Wird jedoch die Liste ***in*** der Liste bearbeitet (Zeile 4), ändert sich auch **`first_copy`** (**'x'** wird auch in die innere Liste **`['c']`** von **`first_copy`** hinzugefügt).

3. deepcopy() (für mutable Objekte)
Soll eine Liste auf jeder Ebene der Verschachtelung kopiert (und damit von der Original-Liste getrennt) werden, muss eine **tiefe Kopie** erzeugt werden. Dabei wird die Datenstruktur (rekursiv) durchlaufen. Zur konkreten Umsetzung kann zum Beispiel **`deepcopy()`** aus der copy-Bibliothek verwendet werden.

```
1  from copy import deepcopy   # importieren der Funktion deepcopy
2
3  original = [['a', 'b'], ['c', 'd']]
4  first_copy = deepcopy(original)
5
6  original[1].append('x')
7  print(original)             # [['a', 'b'], ['c', 'd', 'x']]
8  print(first_copy)           # [['a', 'b'], ['c', 'd']]
```

Alternativ können auch **Arrays aus der NumPy-Programmbibliothek** verwendet werden.

4.2 Tupel

Tupel funktionieren an sich wie Listen mit dem einzigen Unterschied, dass die Datenstruktur **immutable** bzw. der Inhalt eines Tupels nicht änderbar ist. Wurde es einmal erstellt, können einzelne Elemente innerhalb eines Tupels weder hinzugefügt, noch geändert oder gelöscht werden. **Konkatenieren von Tupeln** ist möglich, jedoch wird dabei ein neues Objekt erstellt.

Die **Erstellung eines Tupels** erfolgt wie bei einer Liste jedoch mit **runden Klammern**[16]. Es können beliebig viele Elemente enthalten sein. Bei **Tupeln mit einem Element** ist zu beachten, dass hinter dem Element ein Komma geschrieben werden muss, um es von einem Integer/String/... zu unterscheiden.

[16] Diese könnten an sich sogar weggelassen werden, aber das ist kein guter Stil!

```
tup = (1,2,3)
print(type(tup), id(tup)) # <class 'tuple'> 289[...]008

tup = tup + ('x',)        # ohne das Komma wäre es vom Typ String
print(type(tup), id(tup)) # <class 'tuple'> 289[...]832
```

Wofür werden Tupel benötigt bzw. was ist der Vorteil von einem Tupel gegenüber einer Liste?

Tupel können für das Abspeichern von Daten verwendet werden, die tatsächlich nicht verändert werden sollen. So könnten zum Beispiel Tabellen mit Personendaten oder Konstanten (hier der feste Ort des Punkts `p1` im zweidimensionalen Koordinatensystem) in Form eines Tupels gespeichert werden:

```
musterperson = ('Gabler', 'Erika', 1964, 'Berlin')

p1 = (13.2, -4.55)
```

Die Daten sind dadurch **gegen unbeabsichtigtes Ändern geschützt**.

Außerdem ist das **Laufzeitverhalten von Tupeln besser als das von Listen**. Da ein Tupel nicht aktualisiert werden kann, wird die Datenstruktur statisch in den Speicher geschrieben. Das Auslesen einzelner Werte wird dadurch beschleunigt.

Ein weiterer wichtiger Punkt ist, dass Tupel **als Schlüssel für Dictionary-Objekte** verwendet werden können, Listen jedoch nicht.

4.3 Dictionary

Wie der Name schon vermuten lässt, handelt es sich beim „Dictionary“ um eine Datenstruktur, mit der sich Informationen abspeichern und über einen sogenannten Schlüssel (engl. *key*) leicht wieder finden lassen. Es entspricht also einer **Sammlung von Schlüssel-Objekt-Paaren**. Diese Zuordnung bzw. Abbildung wird auch **Mapping** genannt.

Bei einem klassischen Wörterbuch zum Beispiel entsprechen die Schlüssel den Wörtern in der Ausgangssprache. Durch die alphabetische Sortierung der Schlüssel lässt sich der zugehörige Begriff in der Fremdsprache, also das Objekt, leicht finden.

Auch die Einträge von Dictionaries in Python sind **sortiert**. Allerdings nicht alphabetisch, sondern in der Reihenfolge wie sie eingegeben werden. Einträge (Schlüssel-Objekt-Paare) können **jederzeit gelöscht, geändert oder hinzugefügt** werden. Jeder **Schlüssel** kann **nur einmal vorkommen**, sonst können diese logischerweise nicht eindeutig identifiziert werden. Eine Schlüssel-Dopplung führt jedoch nicht zu einer Fehlermeldung. Der erste Eintrag wird lediglich durch den zweiten überschrieben.

4.3.1 Erstellen von Dictionaries

Eine Liste wird mit eckigen Klammern erstellt und ein Tupel mit runden. Was bleibt somit für das Dictionary übrige? Richtig, **geschweifte Klammern**! Die einzelnen *Schlüssel:Wert*-Paare werden **mit Kommata getrennt** und können der Übersichtlichkeit halber auf mehrere Zeilen geschrieben werden. Hinter das letztes Element *kann* aber *muss* kein Komma gesetzt werden.

```
fahrrad = {                 # Name des Dictionarys
    "typ": "Rennrad",       # Schlüssel: Wert,
    "baujahr": 1978,
    "marke": "PUKY",
    "schaltung": (3, 12)
}
```

Es können beliebige Typen als Werte verwendet werden. Für die **Schlüssel** gilt jedoch, dass **nur Instanzen unveränderlicher (immutable) Datentypen** verwendet werden können (String, Integer, Tupel,...), also beispielsweise keine Listen oder Dictionaries. Ansonsten wird ein TypeError hervorgerufen.

4.3.2 Hilfreiche Operatoren und Funktionen

Zum **Hinzufügen** eines neuen Elements **in ein bereits existierendes Dictionary**, kann folgende Syntax verwendet werden:

```
fahrrad = {
    "typ": "Rennrad",
    "baujahr": 1978
}

fahrrad["preis"] = 99.99

print(fahrrad) # {'typ': 'Rennrad', 'baujahr': 1978, 'preis': 99.99}
```

Existiert der angegebene Schlüssel bereits, wird der alte Wert überschrieben.

Mit der Methode **`update()`** kann man ein zweites Dictionary – aber auch einzelne Elemente – in ein Dictionary einhängen. Auch hierbei werden bereits existierende Werte mit neuen überschrieben.

Außerdem kann es für eine sauber geführte Datenbank interessant sein, aktuell noch vorhandene Datenlücken mit default-Werten zu füllen. Damit ist der Schlüssel bereits im Dictionary vorhanden und kann zu einem späteren Zeitpunkt mit einem sinnvollen Wert gefüllt werden.
Hierfür gibt es die Funktion **`.setdefault(key[, default-Value])`** [17]

[17] Parameter in eckigen Klammern können, müssen aber nicht angegeben werden. Wird hier der `default-Value` weggelassen, wird der Wert des angegebenen Schlüssels auf `None` gesetzt.

Hier ein Beispiel zu `update()` und `setdefault()`:

```
fahrrad = {
    "typ": "Rennrad",
    "marke": "Pukki",
    "baujahr": 1978
}

fahrrad.setdefault("farbe") # setzt "farbe": None

fahrrad_2 = {
    "marke": "PUKY",        # die Reihenfolge der Schlüssel ist egal
    "preis": 99.99,
    "typ": "Kinderrad"
}

fahrrad.update(fahrrad_2)
fahrrad.update({"verfügbar": False})
print(fahrrad)      # {'typ': 'Kinderrad', 'marke': 'PUKY', 'baujahr':
 ↪  1978, 'farbe': None, 'preis': 99.99, 'verfügbar': False}
```

Das **Abrufen der im Dictionary eingespeicherten Werte** erfolgt entweder einzeln über den entsprechenden Schlüssel oder gebündelt per Iteration über Schlüssel **`.keys()`**, Werte **`.values()`** oder Paare **`.items()`**.

Um einen ***KeyError*** durch einen nicht vorhandenen Schlüssel zu verhindern, empfiehlt es sich, vorher dessen Existenz mittels **`if [key] in`** bzw. **`if [key] not in`** zu überprüfen.

```
fahrrad = {
    "typ": "Kinderrad",
    "marke": "PUKY",
    "baujahr": 1978,
}

# print(fahrrad["farbe"])    KeyError! Schlüssel "farbe" nicht vorhanden
if "farbe" in fahrrad:       # Besser!
    print(fahrrad["farbe"])

for x in fahrrad.keys():
    print(x)                 # 'typ' 'marke' 'baujahr'

for x in fahrrad.items():    # x enthält Tupel (key, value)
    # ...
for x,y in fahrrad.items():  # x enthält keys, y enthält values
    # ...
```

Zum **Entfernen einzelner Items** gibt es die Funktionen:

`.pop(key[,default-Wert])` Gibt den Eintrag mit dem Schlüssel `key` zurück und löscht ihn. Durch **Angabe eines default-Werts** wird **kein KeyError** ausgelöst, wenn der Schlüssel nicht vorhanden ist.

`.popitem()` Gibt den zuletzt hinzugefügten Eintrag zurück und löscht ihn.

Mit **`.clear()`** können alle Einträge gelöscht werden, ohne dass das Dictionary selbst gelöscht wird.

```
fahrrad = {
    "typ": "Kinderrad",
    "marke": "PUKY",
    "baujahr": 1978,
    "preis": 99.99,
    "verfügbar": False
}

print(fahrrad.pop("farbe",-1))   # -1  weil farbe nicht in dict vorhanden

print(fahrrad.pop("marke",-1))   # PUKY

print(fahrrad.popitem())         # ('verfügbar', False)

print(fahrrad)  # {'typ': 'Kinderrad', 'baujahr': 1978, 'preis': 99.99}
fahrrad.clear()
print(fahrrad)  # {}
```

Wie bei anderen Datenstrukturen kann auch von einem Dictionary die Länge, also die **Anzahl der Elemente**, über die Funktion **`len()`** abgefragt werden.

```
fahrrad = {
    "typ": "Kinderrad",
    "marke": "PUKY",
    "baujahr": 1978,
    "preis": 99.99,
    "verfügbar": False
}

print(len(fahrrad))      # 5
```

4.3.3 Verschachtelte Dictionaries

Ähnlich wie Listen können auch Dictionaries ineinander verschachtelt sein. Hierbei muss man besonders auf eine korrekte Setzung von Kommata und geschweiften Klammern achten. Die **Tiefe der Verschachtelung** ist auch hier nicht eingeschränkt.

```
bibliothek = {
    "ST250P99 45'3": {
        "titel": "Python",
        "autor_in": ("T. Donaldson",),
        "verlag": "Peachpit Press",
        "jahr": 2014,
        "anzahl_verfuegbar": 28
    },
    "ST250P99 44'5": {
        "titel": "Python 3: lernen und professionell anwenden",
        "autor_in": ("Weigend, M.",),
        "verlag": "Heidelberg; München; Landsberg am Lech u.a.: mitp",
        "jahr": 2013,
        "anzahl_verfuegbar": 6
    },
    "ST250P99 75": {
        "titel": "Programming for Computations - Python",
        "autor_in": ("H. P. Langtangen", "S. Linge"),
        "verlag": "Springer Open",
        "jahr": 2016,
        "anzahl_verfuegbar": 0
    }
}

print(bibliothek["ST250P99 44'5"])          # {'titel': 'Programming
 ↪  for...
print(bibliothek["ST250P99 44'5"]["jahr"]) # 2013
```

4.3.4 Kopieren von Dictionaries

Das Kopieren von Dictionaries funktioniert analog zu Listen. Drei verschiedene Varianten (**=**, **copy()**[18] und **deepcopy()**) werden in Abschnitt 4.1.6 ausführlich beschrieben und lassen sich auf Dictionaries übertragen.

Auch hier die Erinnerung: Da Dictionaries (wie Listen) mutable Objekte sind, muss beim Kopieren von verschachtelten Dictionaries `deepcopy()` verwendet werden. Ansonsten wird von den inneren Dictionaries keine tatsächliche Kopie, sondern nur eine Referenz erzeugt.

[18] [:] funktioniert bei Dictionaries nicht

4.3.5 Umwandlung in eine Liste und umgekehrt

Ein Dictionary lässt sich sehr einfach in eine Liste umwandeln. Über eine der Funktionen **`.keys()`**, **`.values()`** und **`.items()`** muss zunächst eine Auswahl getroffen werden, was in der Liste enthalten sein soll. Über die Funktion **`list()`** wird im nächsten Schritt die Liste gebildet.

Schlüssel und Werte werden in der resultierenden Liste in ihrem jeweiligen Datentypen abgespeichert. Möchte man hingegen die Item-Paare in eine Liste umwandeln, werden diese als Tupel gespeichert.

```
fahrrad = {
    "typ": "Kinderrad",
    "marke": "PUKY",
    "baujahr": 1978,
    "preis": 99.99,
}

vals = fahrrad.values()
vals_list = list(vals)
print(vals_list)         # ['Kinderrad', 'PUKY', 1978, 99.99]

print(list(fahrrad))     # kurz für fahrrad.keys()
                         # ['typ', 'marke', 'baujahr', 'preis']

tuple_list = list(fahrrad.items())  # in einem Schritt
print(tuple_list)        # [('typ', 'Kinderrad'), ('marke', 'PUKY'),
 ↪  ('baujahr', 1978), ('preis', 99.99)]
```

Sollen **zwei Listen in ein Dictionary** umgewandelt werden, werden die einzelnen Wertepaare zunächst mittels **`zip(list_a, list_b)`** zu Tupeln vereint. Dadurch entsteht ein sogenannter Iterator, der auch ausgegeben werden kann.
Achtung: Viele Iteratoren werden bei der Benutzung „verbraucht“. Das bedeutet, dass sie nur einmal durchlaufen werden können:

```
sprachen =  ["C",      "Python", "VHDL"]
varianten = ["else if", "elif", "elsif"]

c = zip(sprachen, varianten)

for i in c:
    print(i)      # ('C','else if'), ('Python','elif'), ('VHDL','elsif')

for i in c:
    print(i)      # Keine Rückgabe, weil Iterator c verbraucht!
```

Vermeidet man ein vorzeitiges Verbrauchen des Iterators, lässt sich aus dem Iterator in einem nächsten Schritt mittels **`dict()`** ein Dictionary erstellen.

```
num = [1, 2, 3, 4]
let = ["a", "b", "c", "d"]

nl_dict = dict(zip(num, let)) # in einem Schritt zusammengefasst
print(nl_dict)                # {1: 'a', 2: 'b', 3: 'c', 4: 'd'}
```

In den bisherigen Beispielen waren beide Listen gleich lang. Dadurch ist bereits eine eindeutige Zuordnung vorhanden. Für den Fall, dass **eine Liste länger** ist als die andere, orientiert sich **`zip()`** an der kürzeren. Die übrigen Werte werden abgeschnitten.

Möchte man hingegen eine Orientierung an der längeren Liste erreichen, kann man die Funktion **`zip_longest()`** (aus der *itertools*-Bibliothek) verwenden. Aber Achtung: `zip_longest()` ergibt natürlich nur Sinn, wenn es mehr Schlüssel als Werte gibt (s. folgendes Beispiel). Den Schlüsseln, zu denen keine Werte angegeben sind, wird ein default-Wert zugewiesen.

```
from itertools import zip_longest

num = [1, 2, 3, 4]
let = ["a", "b"]

print(dict(zip(num, let)))           # {1: 'a', 2: 'b'}
print(dict(zip_longest(num, let)))   # {1: 'a', 2: 'b', 3: None, 4: None}

print(dict(zip_longest(let, num)))   # {'a': 1, 'b': 2, None:4}
```

5 Kontrollstrukturen II

Stellt euch vor, Gauß hätte nie gelebt und ihr sollt ein Programm schreiben, das die Summe der ersten n aufeinander folgenden natürlichen Zahlen bildet. Für $n \leq 5$ könnte das Programm so aussehen:

```
1  nsum = 0              # 'sum' ist in Python bereits vergeben, daher nsum
2  zahl = int(input("Wähle n zwischen 0..5: "))
3  i = 0                 # zählvariable
4
5  if i < zahl:            # 1
6      i += 1              # entspricht i = i + 1
7      nsum += i           # analog nsum = nsum + i
8
9      if i < zahl:            # 2
10         i += 1
11         nsum += i
12
13         if i < zahl:            # 3
14             i += 1
15             nsum += i
16
17             if i < zahl:            # 4
18                 i += 1
19                 nsum += i
20
21                 if i < zahl:            # 5
22                     i += 1
23                     nsum += i
24
25 print("Summe =", nsum)
```

Soll das Programm nun auch für $n = 6$ oder $n = 7$ funktionieren, müsste jedes Mal ein weiterer `if`-Block hinzugefügt werden. Für größere n würde der Code sehr lang und redundant werden. Außerdem würde die Aufgabe ohne die einschränkende Bedingung bezüglich n mit dem bisher gelernten gar nicht umsetzbar sein. Aus diesem Grund gibt es in Python – wie auch in den meisten anderen Programmiersprachen – die **`while`-Schleifen**.

5.1 while-Schleifen

Die `while`-Schleife ist in ihrer Definition leicht verständlich: Der eingerückte Code-Abschnitt wird so lange wiederholt, bis die sogenannte Schleifenbedingung nicht mehr erfüllt ist. Da die Anzahl der Wiederholungen nicht festgelegt ist, wird die `while`-Schleife auch als unbegrenzt bzw. unbeschränkt bezeichnet.

```
# some code before

while x:                  # Schleifenbedingung (logischer Ausdruck)
    # do
    # something

# some code afterwards    wird ausgeführt, sobald (x == False)
```

Wendet man diese Struktur auf das gaußsche Beispiel an, könnte der Code wie folgt aussehen:

```
nsum = 0                  # Initialisieren der Variablen
zahl = int(input("Wähle eine beliebige natürliche Zahl: "))
i = 0

while i < zahl:           # logischer Ausdruck
    i += 1
    nsum += i

print("Summe = ", nsum) # wird ausgeführt, sobald (i < zahl == False)
```

Falls ihr Schwierigkeiten habt, den Ablauf des Programms Schritt für Schritt nachzuvollziehen, könnt ihr den Code bei Spyder eingeben und anschließend das Debugging-Feature (s. Abb. 10) verwenden.

Bevor die Schleife das erste Mal ausgeführt wird, muss jeder Zählvariable (hier **`i`**) und jeder statischen Variable (hier **`zahl`**) ein Wert zugewiesen werden. Auch **`nsum`** muss bereits **vor** der Schleife gleich 0 gesetzt werden, weil die Variable ansonsten mit jedem Schleifendurchlauf wieder zurückgesetzt werden würde, anstatt alle Zahlen wie erwünscht zu addieren.

So wie bei der Variante mit den verschachtelten `if`-Abfragen wird zu Beginn jedes Schleifendurchlaufs überprüft, ob **`i`** immer noch kleiner als **`zahl`** ist. Ist dies der Fall, werden alle direkt folgenden Zeilen ausgeführt, die eingerückt sind (Zeile 6 und 7). Dieser Teil wird auch **Schleifenrumpf** genannt. Sobald beim Start eines neuen Schleifendurchlaufs die Schleifenbedingung nicht mehr erfüllt ist, wird das Programm mit der ersten nicht-eingerückten Zeile fortgesetzt.

Der Vorteil gegenüber den verschachtelten `if`-Abfragen ist jedoch, dass die Schleife mit beliebig großen Zahlen funktioniert und dafür keine Anpassungen vorgenommen werden müssen. Dies ist allgemein ein wichtiges Merkmal von `while`-Schleifen: **Die Anzahl an Iterationen (Schleifendurchläufen) kann beliebig groß sein und muss nicht bekannt sein!**

Es kann immer mal passieren, dass ihr die Schleifenbedingung oder auch den Schleifenrumpf falsch definiert und das Programm die Schleife nicht mehr verlässt. Das bedeutet, dass das Programm nie terminieren wird. In diesem Fall könnt ihr die Ausführung mit der Tastenkombination **Strg + c** manuell abbrechen. In speziellen Fällen ist eine unendliche Ausführung einer Schleife jedoch erwünscht. In diesem Fall kann als Schleifenbedingung **`while (True)`** oder **`while (1)`** gewählt werden, was selbstverständlich niemals „False“ sein kann.

In Verbindung mit der `while`-Schleife gibt es noch zwei Sonderbefehle: `break` und `continue`.

5.1.1 break – Vorzeitiges Verlassen einer Schleife

Wie bereits bekannt ist, wird eine `while`-Schleife regulär beendet, sobald die Schleifenbedingung beim Start eines neuen Schleifendurchlaufs nicht mehr erfüllt ist. Soll die Schleife ***vorzeitig komplett verlassen*** werden, könnt ihr **`break`** verwenden. Wird der `break`-Befehl erreicht, werden die restlichen Befehle des Schleifenrumpfes nicht mehr ausgeführt. Stattdessen läuft das Programm direkt nach der `while`-Schleife weiter.

Im folgenden Beispiel wurde eine Schleifenbedingung gewählt, die immer wahr ist. Daraus folgt, dass die Schleife nur über den `break`-Befehl (Zeile 5) beendet werden kann.

```
1  while True:
2      n = int(input('Gib eine Zahl ein: '))
3
4      if n == 42:
5          break
6      print(f"{n} * {n} = {n*n}")
7
8  print('done')
```

Wenn es für das Programm wichtig ist, ob eine Schleife über die Schleifenbedingung oder einen `break`-Befehl beendet wird, kann hinter die Schleife ein **`else`-Block** hinzugefügt werden. Dieser wird nur ausgeführt, wenn die Schleife nicht über den `break`-Befehl (sondern über die Schleifenbedingung) beendet wurde.

```
1  # Die gesuchte Zahl ist 13.
2  print("5 Versuche, um die gesuchte Zahl zwischen 0 und 15 zu erraten: ")
3  attempt = 0
4
5  while attempt < 5:
6      x = int(input("Eingabe: "))
7      if x == 13:
8          print("Das war richtig!")
9          break
10     print(f"Falsch! Das war der {attempt+1}. Versuch...")
11     attempt += 1
12
13 else:        # wird nur ausgeführt, wenn 5 Mal falsch geraten
14     print("Du hast keinen Versuch mehr übrig!")
```

In Abb. 11a findet ihr ein Flussdiagramm, das die Funktionsweise des `break`-Blocks nochmal veranschaulicht.

5.1.2 continue – Vorzeitiger Abbruch einer Iteration

Mit dem **`continue`-Befehl** wird die aktuelle Iteration (Schleifendurchlauf) unterbrochen, d.h. das Programm kehrt zum Schleifenkopf, also zur Überprüfung der Bedingung, zurück.

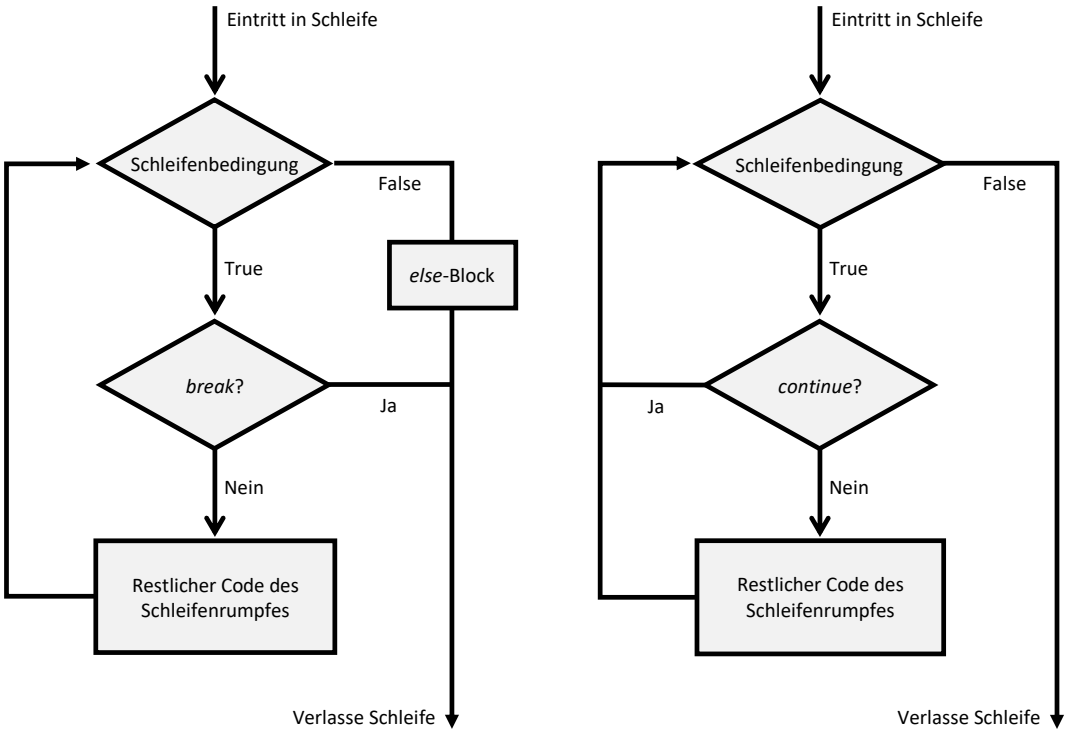
```
1  # jede ungerade Ziffer wird ausgegeben
2  x = 0
3
4  while x < 10:
5      # Quizfrage: Warum muss 'x += 1' vor der if-Abfrage stehen?
6      x += 1
7      if x % 2 == 0:
8          continue
9      print(x)
10
11 # some code afterwards
```

Hierzu findet ihr in Abb. 11b ein Flussdiagramm, das den Ablauf einer Schleife mit einem `continue`-Block zeigt.

5.1.3 Flussdiagramme break und continue

Wie bereits erwähnt, seht ihr hier zwei Flussdiagramme, die den Ablauf eines Programms oder Programmabschnitts vereinfacht visualisieren. Rauten stehen dabei für Verzweigungen bzw. Entscheidungen und Rechtecke für Operationen. Die Pfeile markieren die Richtung zum nächstfolgenden Element.

Der Ablauf wird am Beispiel von **`break`** erklärt:
Beim Erreichen der Schleife wird zunächst die Schleifenbedingung überprüft. Ist die Bedingung erfüllt (True), wird der Schleifenrumpf ausgeführt. Trifft das Programm dabei auf einen `break`-Befehl, wird die Schleife sofort verlassen. Anschließend wird der auf die Schleife folgende Code (ohne `else`-Block!) ausgeführt. Falls der `break`-Befehl nicht erreicht wird, werden alle übrigen Codezeilen des Schleifenrumpfes ausgeführt, eh wieder die Schleifenbedingung überprüft wird.
Sollte die Schleifenbedingung (gleich zu Beginn oder auch in einer späteren Iteration) nicht erfüllt sein, wird die Schleife verlassen. Falls ein zugehöriger `else`-Block geschrieben wurde, wird dieser ausgeführt. Das Programm hat die Schleife durchlaufen und verlässt diese nun.



(a) break (b) continue

Abbildung 11: Vergleich der Flussdiagramme

5.2 for-Schleife

An sich ist die `for`-Schleife ähnlich aufgebaut wie die `while`-Schleife. Es gibt eine Schleifenbedingung, die vor jedem Durchlauf überprüft wird. Ist sie nicht mehr erfüllt, wird die Schleife verlassen. Der große Vorteil einer `for`-Schleife ist jedoch, dass mit ihr **Datenstrukturen durchlaufen** werden können. *Durchlaufen* bedeutet, dass **mit jedem Schleifendurchlauf das nächste Element einer Sequenz** betrachtet wird. Bei der Sequenz kann es sich z.B. um eine Liste, einen String, ein Tupel oder auch ein Dictionary handeln. Diese Datentypen haben nämlich alle die Eigenschaft, dass sie **Iterable** sind, also durchlaufen werden können. In vielen Fällen ergibt sich aus der bekannten Größe der Datenstruktur eine im Vorhinein festgelegte Anzahl an Iterationen (Anzahl Objekte = Anzahl Iterationen).

So sieht die Grundstruktur einer for-Schleife in Python aus:

```
1 for <var> in <iterable>:
2     <statements>
```

<iterable> steht dabei als Platzhalter für die Datenstruktur, die durchlaufen werden soll. Mit jedem Schleifendurchlauf kann über die Variable bei **<var>** auf das aktuelle Objekt zugegriffen werden.

Hier ein konkretes Beispiel: Die Liste `numbers` enthält Zahlen, deren Summe in `result` abgespeichert werden soll.

```
numbers = [2, 5.39, 42, -5, 667, 13]
result = 0.0

for i in numbers:
    result += i

print("Summe =", result)     # Summe = 724.39
```

5.2.1 Stolperfalle: Seiteneffekte von Änderungen an der Datenstruktur

Sofern nicht konkret eine Kopie der Datenstruktur erzeugt wird (s. Abschnitt 4.1.6), muss jedoch beachtet werden, dass alle Änderungen an der Datenstruktur sofort durchgeführt werden und zu unerwarteten Problemen führen können:

```
colors = ["green"]

for c in colors:
    if c == "green":
        colors += ["yellow"]
    if c == "yellow":
        colors += ["green"]

# Diese Zeile würde nie erreicht werden
print(colors)   # ['green', 'yellow', 'green', 'yellow', 'green', ...
```

Dieses Programm terminiert nicht, da im ersten Schleifendurchlauf der String „yellow“ in die Liste hinzugefügt wird. Das neue Element führt dazu, dass die Schleife ein zweites Mal durchlaufen wird und in diesem Schritt der String „green“ in die Liste hinzugefügt wird. Auch dieser führt dazu, dass ein ein weiterer Schleifendurchlauf ausgeführt wird und so weiter. Aus diesem Grund kann es manchmal sinnvoll sein, eine Kopie der Datenstruktur (`colors[:]`) zu verwenden:

```
colors = ["green"]

for c in colors[:]:
    if c == "green":
        colors += ["yellow"]
    if c == "yellow":
        colors += ["green"]

print(colors)   # ['green', 'yellow']
```

5.2.2 break und continue

Auch bei der `for`-Schleife können `break`- und `continue`-Befehle zur Steuerung des Programmablaufs eingebaut werden. Das Verhalten ist identisch zu ihrer Verwendung in `while`-Schleifen und wird daher an dieser Stelle nicht erneut erläutert. Nur ein kurzer Hinweis: Bei `continue` wird der aktuelle Schleifendurchlauf abgebrochen und die Iteration mit dem **nächsten** Element der Liste fortgesetzt.

5.2.3 range()-Funktion

Mit der range()-Funktion lassen sich Integer-Sequenzen erstellen. Wie der Name schon vermuten lässt, wird über die Argumente ein Bereich angegeben, in dem sich die Werte befinden. Es können bis zu drei Parameter übergeben werden:

start *(optional)*	Startwert der Sequenz. Ist selber enthalten! (default = 0)
stop	Endwert der Sequenz. Ist selber nicht enthalten!
step *(optional)*	Schrittweite (default = 1)

Das Ergebnis wird der Effizienz wegen seit Python 3 nicht mehr als Liste sondern als „range object“ gespeichert. Daraus lässt sich aber mittels `list()` eine Liste erstellen.

```
1 list(range(10))          # [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
2 list(range(4, 44, 8))    # [4, 12, 20, 28, 36]
```

In Verbindung mit der `for`-Schleife ist diese Funktion sehr hilfreich, da sich ein „range object“ als Iterable verwenden lässt und auf diese Weise über große Zahlensequenzen iteriert werden kann.

So lässt sich zum Beispiel auch die Summe der ersten n aufeinander folgenden natürlichen Zahlen mit einer `for`-Schleife berechnen:

```
1 nsum = 0                    # Initialisieren der Variablen
2 n = int(input("Wähle eine beliebige natürliche Zahl: "))
3
4 for i in range(1, n+1):   # n+1 weil Endwert nicht mehr enthalten
5     nsum += i
6
7 print("Summe =", nsum)
```

6 Funktionen

In der Programmierung müssen die gleichen Codeabschnitte oft an mehreren Stellen im Programm ausgeführt werden. Natürlich könnte man diese Codeabschnitte kopieren und an verschiedenen Stellen des Programms wieder einfügen, doch dies senkt die Lesbarkeit und Wartbarkeit eures Codes[19]. Es gibt eine deutlich elegantere Lösung – die Verwendung von Funktionen.

6.1 Was sind Funktionen?

Eine Funktion verpackt einen Codeabschnitt im sogenannten Funktionsrumpf und gibt ihm einen Namen. Über diesen Namen kann der Codeabschnitt dann beliebig oft aufgerufen werden. Außerdem kann man einer Funktion Werte übergeben („Argumente“), die der Codeabschnitt zur Ausführung benötigt. Funktionen können ebenfalls Werte zurückgeben („Returnwerte“), welche zum Beispiel verwendet werden können, um den weiteren Programmfluss zu beeinflussen. In Abschnitt 6.2 wird dies durch Beispiele näher erläutert.

6.2 Wie verwendet man Funktionen?

Die einfachste Form einer Funktion wird allein durch einen Funktionsnamen und einen Codeabschnitt definiert. Im nachfolgenden Beispiel wird eine Funktion mit dem Namen **`sample_function`** definiert. Definieren bedeutet hier, dass die Funktion im auf die Definition folgenden Code bekannt ist. Sie muss allerdings explizit aufgerufen werden, damit der Codeabschnitt ausgeführt wird.

```
1 def sample_function():
2     print("some text")
```

Das Keyword *def* gibt hierbei an, dass eine Funktion *def*iniert wird. Anschließend folgt der Name der Funktion, welche definiert werden soll. Auf den Funktionsnamen folgen runde Klammern. Für komplexere Funktionen, welche Eingabewerte („Argumente”) verarbeiten, bleiben die Klammern nicht leer. Dies wird in späteren Beispielen verdeutlicht. Schlussendlich wird die Zeile durch einen Doppelpunkt abgeschlossen. Der Code, der beim Aufruf der Funktion ausgeführt werden soll, wird – um eine Ebene eingerückt – in die folgenden Zeilen geschrieben. Nachfolgende Zeilen, welche nicht eingerückt sind, sind nicht mehr Teil der Funktion.

Will man die Funktion aufrufen, muss an einer beliebigen Stelle der Funktionsname gefolgt von runden Klammern eingefügt werden. Als einzige Bedingung gilt, dass die Funktion vor ihrem ersten Aufruf bereits bekannt sein, also sich im Code „weiter oben“ befinden muss.

```
1 sample_function()
```

Ein Beispielskript, welches eine Funktion **`print_numbers()`** definiert und aufruft, könnte wie folgt aussehen. Dabei werden bei Aufruf der Funktion die Zahlen 0-5 ausgegeben.

[19] Daher soll dies in euren kommenden Abgaben auch nicht mehr vorkommen!

```
1 def print_numbers():
2     for x in range(6):
3         print(x)
4
5 print_numbers()
```

Als nächstes schauen wir uns eine Funktion an, welche zwei Zahlen addiert. Anders als die Funktion **`print_numbers()`** benötigt diese Funktion zwei Übergabewerte („Argumente"), nämlich die beiden Summanden. Die Funktion könnte wie folgt implementiert und aufgerufen werden:

```
1 def add(a, b):
2     total = a + b
3     print("Summe:", total)
4
5 add(4, 2)
```

Betrachten wir zunächst Zeile 5. Hier wird die Funktion **`add`** mit den Argumenten 4 und 2 aufgerufen. Innerhalb der Funktion werden Argumente als sogenannte Parameter empfangen. Man ruft eine Funktion also mit Argumenten auf, und arbeitet innerhalb der Funktion mit Parametern. Es handelt sich lediglich um eine unterschiedliche Bezeichnung. In Zeile 1 wurden die Parameter **`a`** und **`b`** in runden Klammern und durch Kommata getrennt definiert. Unter Berücksichtigung der Reihenfolge wird nun **`a=4`** sowie **`b=2`** gesetzt. Innerhalb des Codeblocks („Funktionsrumpf") der Funktion sind **`a`** und **`b`** nun Variablen, welche wie gewohnt verwendet werden können. Bei Aufruf der Zeile 5 wird **`Summe: 6`** über die Konsole ausgegeben.

Will man die Summe im weiteren Programmverlauf weiterverwenden, so sollte der Wert als Rückgabewert („Returnwert") zurückgegeben werden. Dies ist über das **`return`** Keyword möglich. Bei erreichen dieses Schlüsselwortes wird die Ausführung der Funktion abgebrochen (terminiert). Die nachfolgenden Werte werden zurückgegeben.

```
1 def add(a, b):
2     total = a + b
3     return total
4
5 calculation_result = add(4, 2)
6 print("Summe:", calculation_result)
```

In Zeile 5 wird die Funktion **`add`** mit den Argumenten 4 und 2 aufgerufen. Der Wert, der in Zeile 3 zurückgegeben wird, landet in der Variable **`calculation_result`**, ebenfalls in Zeile 5. Auf niedrigerer Ebene wird zunächst **`add(4, 2)`** zu **`6`** aufgelöst, und dann der Variable **`calculation_result`** zugewiesen.

So wie eine Funktion viele Parameter akzeptieren kann (maximal 255), so kann sie auch viele Werte zurückgeben. Hierzu werden die Werte hinter dem **`return`**-Keyword durch Kommata getrennt aufgelistet. Im folgenden Beispiel wird zusätzlich zu der Summe der beiden Parameter eine Variable

greater_or_equal_to_zero vom Typ **bool** zurückgegeben. Sie enthält, wie der Name schon vermuten lässt, den Wert **True**, wenn die Summe größer oder gleich 0 ist. Anderenfalls enthält sie den Wert **False**.

```
1  def add(a, b):
2      total = a + b
3      greater_or_equal_to_zero = total >= 0
4      return total, greater_or_equal_to_zero
5
6  calculation_result, not_negative = add(4, 2)
7  print("Summe:", calculation_result, "Größer gleich 0:", not_negative)
```

In Zeile 6 sollte der zweite Rückgabewert nun, so wie die Summe selbst, ebenfalls in einer Variable gesichert werden. Im Beispiel wird der Rückgabewert **True** in der Variable **not_negative** gesichert.

Das **return**-Keyword kann auch in Funktionen ohne Rückgabewerte verwendet werden. In diesem Fall steht es alleine und beendet die Ausführung der Funktion frühzeitig. Dies, sowie der Aufruf einer Funktion mit Variablen anstelle von expliziten Werten, wird im folgenden Beispiel verdeutlicht.

```
1  def add_and_print_non_negative_numbers(a, b):
2      if a < 0 or b < 0:
3          return
4      total = a + b
5      print(total)
6
7  first_number = 4
8  second_number = 0
9  add_and_print_non_negative_numbers(first_number, second_number)
```

In diesem Beispiel wird die Ausführung der Funktion **add_and_print_non_negative_numbers** frühzeitig beendet, falls **a** oder **b** negativ sind.

Generell können die Datentypen von Argumenten und Rückgabewerten beliebig gemischt werden. Es kann also z.B. ein Integer und ein String übergeben werden, und ein Boolean sowie ein Float zurückgegeben werden. Beim Aufrufen der Funktion ist stets auf die richtige Reihenfolge der Argumente zu achten, da es sonst zu Problemen kommen kann.

6.3 Naming Conventions

Bei der Benennung von Funktionen gelten die gleichen Richtlinien wie bei der Benennung von Variablen. Grundsätzlich werden Funktionsnamen klein geschrieben. Besteht der Name einer Funktion aus mehreren Teilen, werden diese durch einen Unterstrich getrennt. Soll eine Funktion also eine Nachricht versenden, wäre der Name **send_message** anderen Möglichkeiten wie **sendmessage**, **sendMessage** oder **SendMessage** vorzuziehen.

6.4 Sichtbarkeitsbereich von Variablen

Nachdem ihr verstanden habt, was Variablen und Funktionen sind, ist es wichtig den Sichtbarkeitsbereich von Variablen („Variable Scope") zu verstehen. Definierte Variablen sind und sollen in vielen Fällen auch nicht überall sichtbar sein.

Die Sichtbarkeit einer Variable beschreibt, welche Teile vom Code auf sie zugreifen können. Man kann eine sichtbare Variable z.B. über ein **`print()`** Statement über die Konsole ausgeben.

Auf oberster Ebene definierte Variablen, sogenannte globale Variablen, welche also nicht innerhalb einer Funktion definiert wurden, sind im gesamten Skript sichtbar. Ist euer Programm in mehrere Dateien aufgeteilt, müsst ihr in anderen Dateien befindliche Variablen über den Import-Namen referenzieren (**`{import-name}.{variablen-name}`**). Mehr Informationen dazu findet ihr in Abschnitt 7.1.1.

Innerhalb einer Funktion definierte Variablen, sogenannte lokale Variablen, sind im Gegensatz dazu nur innerhalb dieser Funktion sichtbar. Folgendes Beispiel zeigt, dass global definierte Variablen auch innerhalb von Funktionen sichtbar sind.

```
1 my_variable = "Globaler Wert"
2
3 def my_function():
4     print(my_variable)
5
6 my_function()
```

Führt man dieses Skript aus, wird über die Konsole der Text „Globaler Wert" ausgegeben.

Folgendes Beispiel zeigt, dass lokal definierte Variablen nur innerhalb der Funktion sichtbar sind.

```
1 def my_function():
2     my_variable = "Lokaler Wert"
3     print(my_variable)
4
5 my_function()
6 print(my_variable)
```

Führt man das Skript aus, wird zunächst der Text „Lokaler Wert" über die Konsole ausgegeben. Bei Ausführung der Zeile 6 kommt es allerdings zu einem Fehler, da die Variable **`my_variable`** an dieser Stelle nicht bekannt ist.

Zusätzlich ist es wichtig zu verstehen, dass globale Variablen von lokalen Variablen überschrieben werden. Definiert man in einer Funktion eine Variable, deren Name schon außerhalb der Funktion existiert, so greift man innerhalb der Funktion immer auf die lokal definierte Variable zu. Für Zugriffe von außerhalb der Funktion ändert sich allerdings nichts. Folgendes Beispiel verdeutlicht dieses Verhalten.

```
my_variable = "Globaler Wert"

def my_function():
    my_variable = "Lokaler Wert"
    print("Variable in der Funktion:", my_variable)

my_function()
print("Variable außerhalb der Funktion:", my_variable)
```

Die Ausgabe dieses Skripts ist wie folgt:

```
> Variable in der Funktion: Lokaler Wert
> Variable außerhalb der Funktion: Globaler Wert
```

6.5 Nützliche Funktionen

Python bietet eine Reihe von integrierten Funktionen, welche im alltäglichen Gebrauch sehr nützlich sind.

Tabelle 3: Nützliche Funktionen I

Funktion	Beschreibung	Beispiel
len()	Gibt die Anzahl von Zeichen in einem String oder die Anzahl von Elemente in einer Liste zurück.	`print(len("Hello World")) # 11` `print(len(["a", "b", "c"])) # 3`
upper()	Konvertiert einen String in Großbuchstaben.	`print("aBc123!".upper()) # "ABC123!"`
lower()	Konvertiert einen String in Kleinbuchstaben.	`print("aBC123!".lower()) # "abc123!"`
strip()	Entfernt Whitespace am Anfang und Ende eines Strings.	`print("  abc  ".strip()) # "abc"`
replace()	Ersetzt Teile eines Strings. Der erste Parameter definiert den zu ersetzenden Teilstring, der zweite Parameter definiert den String, der stattdessen eingesetzt wird. Der Teilstring wird an jeder Stelle im String ersetzt, wo er vorkommt.	`print("Hello world!".replace("l", "_")) # "He__o wor_d!"` `print("Hello world!".replace("world", "you")) # "Hello you!"`
split()	Teilt einen String basierend auf einem Trennzeichen in mehrere Teilstrings auf und verpackt diese als Liste.	`print("Hello world!".split(" ")) # ['Hello', 'world!']`
join()	Verbindet die Elemente einer Liste zu einem String, getrennt durch ein definiertes Trennzeichen.	`print(";".join(["a", "b", "c"])) # "a;b;c"`

Tabelle 4: Nützliche Funktionen II

Funktion	Beschreibung	Beispiel
abs()	Entfernt das Vorzeichen einer Zahl.	`print(abs(-1.33)) # 1.33`

round()	Rundet eine Zahl auf die nächste ganze Zahl.	`print(round(1.4)) # 1`
		`print(round(1.5)) # 2`
		`print(round(-1.5)) # -2`

min()	Gibt das kleinste Element zurück. Bei einer Liste von Buchstaben sowie bei Strings wird der im Alphabet am frühesten auftretende Buchstabe zurückgegeben.	`print(min(2, 3, 1)) # 1`
		`print(min("b", "a", "c")) # "a"`
		`print(min(["b", "a", "c"])) # "a"`
		`print(min("helloworld")) # "d"`

max()	Verhält sich gegenteilig zu **`min()`**.	`print(max(2, 3, 1, -3, 8)) # 8`
		`print(max("b", "a", "c")) # "c"`
		`print(max(["b", "a", "c"])) # "c"`
		`print(max("helloworld")) # "w"`

sum()	Summiert die Elemente einer Liste auf und gibt die Summe zurück.	`print(sum([1, 2, 3, 4, -8])) # 2`
count()	Zählt, wie oft ein gegebenes Element in einer Liste vorkommt.	`print(["a", 13, -2.5, False, "a",` `↪ "b"].count("a")) # 2`

type()	Gibt den Typ einer Variablen zurück.	`print(type(1)) # int`
		`print(type(1.5)) # float`
		`print(type("Hello world!")) # str`
		`print(type(True)) # bool`

6.6 Type Casting

Type casting, auch als „Typumwandlung" bezeichnet, beschreibt das Konvertieren einer Variable zu einem anderen Datentypen. Dies wird erreicht, indem der zu konvertierende Wert in Klammern gesetzt wird, mit dem gewünschten Datentypen vorneweg.

```
my_variable = 13
print(my_variable) # 13
print(type(my_variable)) # int

my_variable = str(my_variable)

print(my_variable) # "13"
print(type(my_variable)) # str
```

Versucht man allerdings zum Beispiel einen String in einen Integer umzuwandeln, so ist dies nicht bedingungslos möglich.

```
int("13") # OK
int("abc") # ERROR
```

Ein konkreter Anwendungsfall, bei dem ihr type-casting benötigen werdet, ist das Konkatenieren von Strings mit Zahlen. Die folgende Codezeile kann nicht erfolgreich ausgeführt werden:

```
print("test" + 123)
```

Castet man die Zahl jedoch erst zu einem String, geht alles gut.

```
print("test" + str(123)) # "test123"
```

In Abschnitt 3.1.3 findet ihr einen weiteren üblichen Anwendungsfall für type-casting – die manuelle Eingabe von Daten.

6.7 Rekursion

Rekursion bezeichnet den Fall, dass eine Funktion sich, direkt oder indirekt, selbst aufruft. Dies erleichtert die Lösung von komplexen Probleme oft enorm, hat aber auch nicht zu vernachlässigende Nachteile – zum Beispiel der deutlich höhere Gebrauch von Arbeitsspeicher. Das Thema ist nicht Teil dieses Kurses. Sollte allerdings trotzdem Interesse bestehen, könnt ihr euch hier weiter über das Thema informieren: `https://www.python-kurs.eu/rekursive_funktionen.php`

7 Module, Dateioperationen, Plotting, Exceptions

Im diesem Abschnitt werden euch verschiedene Themen näher gebracht, welche gerade für einen Messversuch, wie ihr ihn bald durchführen werdet, unerlässlich sind. Mit dem Wissen dieses Kapitels werdet ihr in der Lage sein Module zu verwenden sowie eigene Module zu erstellen — dies ermöglicht es euch, euer Programm auf verschiedene Dateien aufzuteilen. Zusätzlich lernt ihr, Informationen aus Dateien zu verarbeiten und Daten in Dateien zu speichern, Messergebnisse als professionelle Graphen zu plotten, sowie mit Fehlern bei der Programmausführung umzugehen, sodass euer Programm bei Fehlern nicht einfach abstürzt.

7.1 Module

Je größer unsere Python-Skripte werden, desto unübersichtlicher werden sie. Ab einer bestimmten Größe macht es Sinn, Skripte in mehrere Dateien aufzuteilen. Die einzelnen **`.py`** Dateien nennt man Module. Man kann Module mittels des **`import`** Schlüsselwortes in andere Skripte einbinden und definierte Funktionen aufrufen und definierte Variablen referenzieren.

7.1.1 Eigenes Modul erstellen

Im folgenden Beispiel wird gezeigt, wie ein eigenes Modul aussehen kann. Das Modul stellt die Grundrechenarten mittels Funktionen zur Verfügung. Wir nennen unsere Datei **`calculator.py`**.

```
1  def add(a, b):
2      return a + b
3
4  def subtract(a, b):
5      return a - b
6
7  def multiply(a, b):
8      return a * b
9
10 def divide(a, b):
11     return a / b
```

Nun wollen wir unser Skript in einem anderen Skript einbinden. Das folgende Vorgehen funktioniert nur, wenn sich die beiden Skripte **im gleichen Ordner befinden**. Mithilfe des **`import`** Schlüsselwortes, gefolgt vom Modulnamen wird ein Skript in ein anderes Skript eingebunden. Der Name des Moduls entspricht dem Dateinamen ohne Dateiendung. In unserem Fall heißt unser Modul also **`calculator`**. Die Funktionen des Moduls können anschließend mit **`<modulname>.<funktionsname>(<argumente>)`** verwendet werden.

```
import calculator

print(calculator.add(3, 2)) # 5
print(calculator.subtract(3, 2)) # 1
print(calculator.multiply(3, 2)) # 6
print(calculator.divide(3, 2)) # 1.5
```

Wir können dem importierten Modul bei der Einbindung einen Nicknamen geben, was uns Tipparbeit abnehmen und zur Lesbarkeit des Codes beitragen kann. Im nachfolgenden Beispiel geben wir dem Modul **`calculator`** den Nicknamen **`calc`** mittels der Konstruktion **`import [modulname] as [nickname]`**.

```
import calculator as calc

print(calc.add(3, 2)) # 5
print(calc.subtract(3, 2)) # 1
print(calc.multiply(3, 2)) # 6
print(calc.divide(3, 2)) # 1.5
```

7.1.2 Beispiel math Modul

Eine große Stärke von Python ist, dass es unzählige, kostenfreie Module gibt, welche ihr in euren Skripten verwenden könnt. Bekannte Beispiele sind **`numpy`**, **`scipy`** und **`pandas`**. Es ist zu beachten, dass diese unter Umständen zunächst installiert werden müssen[20].

Als Beispiel wollen wir euch ein paar Funktionen des **`math`** Moduls zeigen. Dieses bietet deutlich mehr Funktionen als unser eigenes **`calculator`** Modul.

Im nachfolgenden Skript werden drei Funktionen des **`math`** Moduls verwendet. Die Funktion **`sin()`** nimmt einen Float x als Parameter und gibt $sin(x)$ zurück. Im Beispiel wird $sin(\frac{\pi}{2}) = 1$ berechnet. Dabei wird die Konstante **`pi`** des **`math`** Moduls verwendet, sodass der tatsächliche Wert von π nicht eingegeben werden muss. Die Funktion **`log()`** berechnet den Logarithmus einer Zahl zu einer beliebigen Basis. Dazu werden die Zahl sowie die Basis als Argumente übergeben. Im Beispiel wird $log_{10}(100) = 2$ berechnet. Die Funktion **`factorial()`** des **`math`** Moduls berechnet die Fakultät einer Zahl. Im Beispiel wird $5! = 1 \cdot 2 \cdot 3 \cdot 4 \cdot 5 = 120$ berechnet.

```
import math

print(math.sin(0.5 * math.pi)) # 1.0
print(math.log(100, 10)) # 2.0
print(math.factorial(5)) # 120
```

[20] Eine Anleitung für die Installation von Modulen kann hier gefunden werden: `https://packaging.python.org/tutorials/installing-packages/`

7.2 Dateioperationen

Als Nächstes befassen wir uns mit dem Thema Dateioperationen. Es umfasst das Lesen, Schreiben und Verarbeiten von Dateien.

Unabhängig davon, ob wir lesen oder schreiben wollen, muss die entsprechende Datei zunächst geöffnet werden. Dies ist mit der von Python standardmäßig angebotenen Funktion **`open()`** möglich. Hierbei wird als erstes Argument der Dateiname angegeben. Liegt die Datei nicht im gleichen Ordner wie die **`.py`** Datei, so kann alternativ auch der exakte Pfad der Datei angegeben werden. Das zweite, optionale Argument gibt an, in welchem Modus die Datei geöffnet wird. Uns interessieren die Optionen **`r`** für *read* und **`w`** für *write*. Lässt man das zweite Argument weg, wird die Datei im *read* Modus geöffnet.

Wir beschränken uns auf die Verarbeitung von Dateien welche Text enthalten. Binärdateien lassen sich mit Python allerdings ebenfalls verarbeiten.

Damit wir mit der Datei in den folgenden Zeilen arbeiten können, speichern wir, vereinfacht gesagt, die Datei in einer Variablen. Im folgenden Beispiel ist es die Variable **`f`**. Zusätzlich sollte das Encoding (Zeichenkodierung) angegeben werden. Das Standardencoding unter Windows und Linux ist unterschiedlich. Damit das Skript sich auf beiden Betriebssystemen gleich verhält, kann das Encoding explizit angegeben werden. In den folgenden Beispielen wird das Encoding **`utf-8`** verwendet. Wir empfehlen euch, dieses ebenfalls zu verwenden.

```
1 f = open("some_file.txt", "r", encoding="utf-8")
```

Wie euch vielleicht aufgefallen ist, wird der String **`utf-8`** nicht einfach als simples Argument übergeben. Es handelt sich um ein sogenanntes *Keyword Argument*. Funktionen, welche Keyword Arguments akzeptieren, können die Werte nach dem vorangegangenen Keyword, in diesem Fall **`encoding`** zuordnen. Normale Argumente werden, wie ihr bereits gelernt habt, über die Reihenfolge zugeordnet. In folgenden Beispielen werdet ihr auf weitere Keyword Arguments stoßen. Jetzt wisst ihr, was es damit auf sich hat.

Es ist äußerst wichtig, geöffnete Dateien mit der **`close()`** Funktion auch wieder zu schließen, damit blockierte Ressourcen wieder freigegeben werden. Der Zugriff auf bereits geschlossene Dateien führt zu einer Fehlermeldung.

```
1 f.close()
```

7.2.1 Dateien lesen

Nun können wir über die Funktion **`read()`** den Inhalt der Datei auslesen und diesen beispielsweise über die Konsole ausgeben.

```
1 f = open("some_file.txt", "r", encoding="utf-8")
2 print(f.read()) # Gibt den Inhalt der Datei "some_file.txt" über die
  ↪ Konsole aus
3 f.close()
```

Oft enthalten Dateien geordnete Informationen, zum Beispiel einen Datensatz pro Zeile. Will man die Zeilen einer Datei separat behandeln, kann man die Methode[21] **`splitlines()`** auf dem von **`read()`** zurückgegebenen String ausführen[22]. Man erhält eine Liste von Strings. Jeder String in der Liste entspricht einer Zeile der Datei. Im folgenden Beispiel wird über die Liste iteriert und jede Zeile separat über die Konsole ausgegeben.

```
f = open("some_file.txt", encoding="utf-8")
for line in f.read().splitlines():
    print(line)
f.close()
```

Fehlerresistente Variante zum Öffnen einer Datei: Geht nach dem Öffnen der Datei etwas schief, ist nicht garantiert, dass die Datei auch wieder geschlossen wird. Das **`with`** Statement stellt das korrekte Öffnen und Schließen von Dateien sicher. Alle Operationen, die an der Datei durchgeführt werden sollen, müssen (wie zum Beispiel bei einer Schleife) in den folgenden Zeilen um eine Ebene eingerückt werden.

```
with open("some_file.txt", encoding="utf-8") as f:
    for line in f.read().splitlines():
        print(line)

# Beispiel für Dateizugriff, nachdem die Datei schon geschlossen wurde
↪  (da nicht mehr eingerückt)
f.read()    # ValueError: I/O operation on closed file.
```

7.2.2 Dateien schreiben

Wie bereits erwähnt, lassen sich Dateien mittels Python auch schreiben. Hierfür muss die Datei im *write* Modus geöffnet werden. Anschließend kann man Text über die **`write()`** Methode einfügen. Im folgenden Beispiel werden drei Zeilen in eine neue Datei **`output.txt`** geschrieben.

```
with open("output.txt", "w", encoding="utf-8") as f:
    f.write("Zeile 1\n")
    f.write("Zeile 2\n")
    f.write("Zeile 3\n")
```

Das **`\n`** am Ende der Zeilen ist ein Zeilenumbruch. Wenn man diesen weglässt, werden die drei Strings in eine Zeile hintereinander geschrieben. Die vom obigen Skript generierte Datei sieht wie folgt aus.

[21] Eine Methode ist eine Funktion eines Objekts. Mehr dazu in Abschnitt 8.

[22] Alternativ könnt ihr euch die Methode **`readlines()`** anschauen, welche direkt auf **`f`** ausgeführt wird. Beachtet allerdings, dass hier das **`\n`** am Ende der Zeilen nicht entfernt wird. In unserem Beispiel käme es also bei der Ausgabe zu Leerzeilen.

```
Zeile 1
Zeile 2
Zeile 3

```

Es ist wichtig zu wissen, dass das obige Beispiel eine eventuell existierende Datei mit dem gleichen Namen überschreibt. Will man stattdessen eine existierende Datei erweitern, so kann man die Datei mit **a** als zweites Argument im *append* Modus anstelle vom *write* Modus öffnen.

7.2.3 Umgang mit CSV-Dateien

Oft werden organisierte Daten in CSV Dateien tabellenartig abgelegt. Dabei steht jeder Datensatz in einer eigenen Zeile, und jede Spalte ist durch ein Trennzeichen („Delimiter") abgegrenzt. Nachfolgend werden wir die Datei **ages.csv**, deren Inhalt im folgenden gezeigt ist, betrachten. Die erste Spalte enthält Namen von Studierenden, während die zweite Spalte deren Alter enthält. Getrennt werden die Spalten durch Semikola.

```
Max;21
Sabine;19
Jonas;28
Jakob;19
Sarah;23
```

Für die einfache Verarbeitung von CSV Dateien verwenden wir das Modul **csv**. Mittels der **reader()** Funktion, an die wir unsere geöffnete Datei sowie das Trennzeichen der CSV Datei übergeben, können wir auf die Daten auf simple Weise zugreifen. Iterieren wir über das von der Funktion zurückgegebene Objekt in einer Schleife, so entspricht jedes Element einer Zeile in der CSV Datei. Die Informationen einer Zeile sind als Liste hinterlegt, deshalb können wir, wie bei jeder Liste, gezielt auf bestimmte Elemente zugreifen. Dabei entspricht das Element an Index 0 dem Eintrag der ersten Spalte.

```
import csv

with open("ages.csv", "r", encoding="utf-8") as f:
    reader = csv.reader(f, delimiter=";")
    for row in reader:
        print(row[0], "ist", row[1], "Jahre alt.")
```

Das obige Beispiel erzeugt folgende Ausgabe über die Konsole:

```
Max ist 21 Jahre alt.
Sabine ist 19 Jahre alt.
Jonas ist 28 Jahre alt.
Jakob ist 19 Jahre alt.
Sarah ist 23 Jahre alt.
```

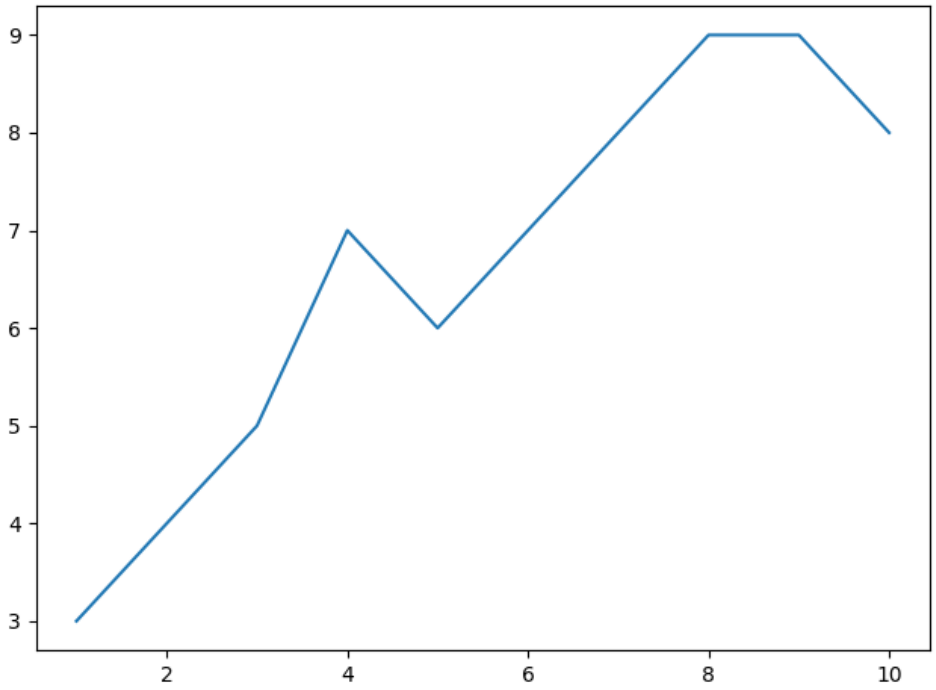
7.3 Graphen plotten

In diesem Abschnitt wollen wir euch die Python Bibliothek[23] **`matplotlib`** vorstellen. Sie ermöglicht es, professionelle Diagramme mit geringem Aufwand zu generieren. Wir binden **`matplotlib.pyplot`** ein. **`pyplot`** ist eine Schnittstelle von **`matplotlib`** und erlaubt die Generierung von Graphen auf die selbe, simple Art und Weise wie in der Programmiersprache MATLAB.

Nach der Einbindung der Bibliothek ist der nächste Schritt, die Daten für das Plotting vorzubereiten. Wir benötigen mindestens zwei Listen, welche zusammen den Graphen beschreiben. Eine Liste enthält die x-Werte, die andere Liste die y-Werte. Werte an einem bestimmten Index in beiden Listen bilden zusammen einen Datenpunkt. Die Listen werden anschließend an die **`plot()`** Funktion der Bibliothek übergeben. Dabei entspricht das erste Argument den x-Werten und das zweite Argument den y-Werten. Mit Aufruf der **`show()`** Funktion kann man sich den Graphen anschließend im *Abbildungen*-Reiter (s. Abb. 10 mit der Nummer 3) anzeigen lassen.

```
1 import matplotlib.pyplot as plt
2
3 x = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
4 y = [3, 4, 5, 7, 6, 7, 8, 9, 9, 8]
5
6 plt.plot(x, y)
7 plt.show()
```

Der obige Code produziert den nachfolgenden Graphen.

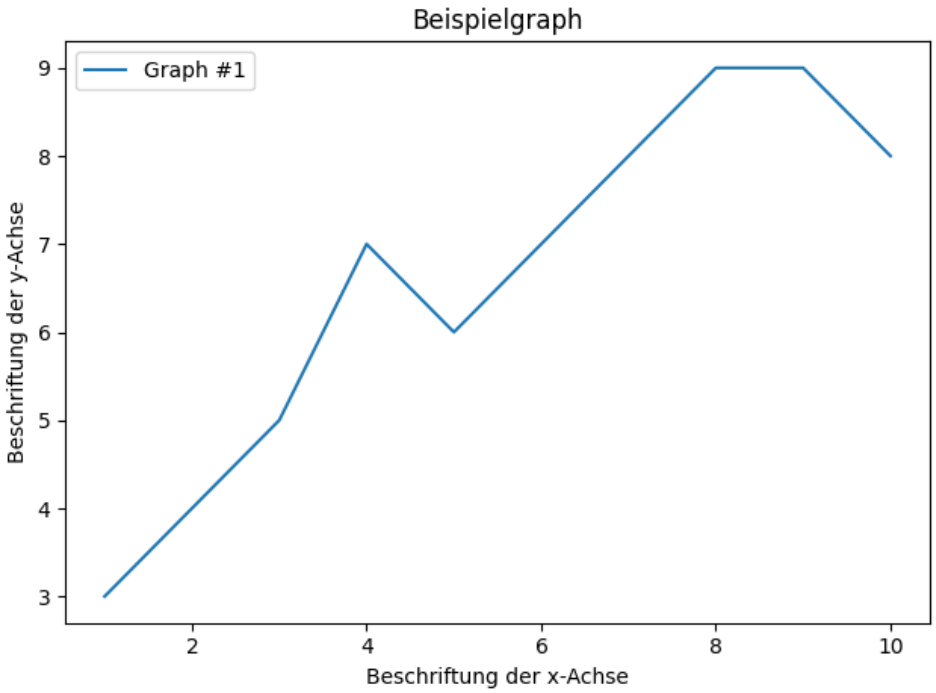


[23] Als Bibliothek bezeichnet man in Python eine Sammlung von Modulen.

Natürlich soll aus dem Graphen auch erkennbar sein, welche Informationen enthalten sind und welche Werte auf den Achsen abgebildet sind. Der **`plot()`** Funktion kann, wie in Zeile 6 des folgenden Beispiels erkennbar, über das „Keyword Argument“ **`label`** ein Label übergeben werden, welches im Graphen einen Legendeneintrag anlegt. Damit dies funktioniert, muss zusätzlich vor der **`show()`** Funktion die **`legend()`** Funktion der Bibliothek aufgerufen werden. Die Achsen lassen sich über die Funktionen **`xlabel()`** und **`ylabel()`** beschriften. Dem ganzen Graphen kann mit der **`title()`** Funktion ein Titel gegeben werden.

```
1  import matplotlib.pyplot as plt
2
3  x = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
4  y = [3, 4, 5, 7, 6, 7, 8, 9, 9, 8]
5
6  plt.plot(x, y, label="Graph #1")
7
8  plt.xlabel("Beschriftung der x-Achse")
9  plt.ylabel("Beschriftung der y-Achse")
10 plt.title("Beispielgraph")
11 plt.legend()
12
13 plt.show()
```

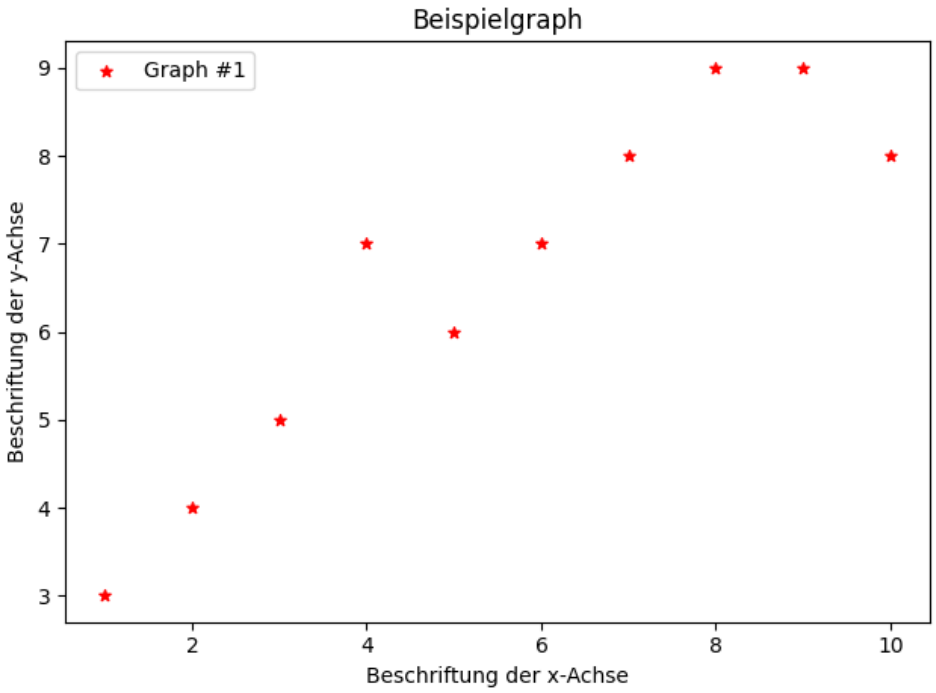
Dies ist der vom obigen Codeabschnitt generierte Graph.



Auch diskrete Werte können abgebildet werden. Es muss lediglich die **`scatter()`** Funktion anstelle der **`plot()`** Funktion verwendet werden. Über das Keyword Argument **`marker`** kann die Form der Datenpunkte im Graphen bestimmt werden. Im folgenden Beispiel wurde die Sternform gewählt, aber es gibt weitere Formen[24] zur Auswahl. Experimentiert gerne mit den verschiedenen Formen. Das Keyword Argument **`s`** definiert die Größe der Datenpunkte im Graphen. Das Keyword Argument **`color`** erlaubt es, die Farbe der Datenpunkte zu bestimmen. Dies ist auch bei normalen Plots möglich.

```
1  import matplotlib.pyplot as plt
2
3  x = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
4  y = [3, 4, 5, 7, 6, 7, 8, 9, 9, 8]
5
6  plt.scatter(x, y, label="Graph #1", color="red", marker="*", s=30)
7
8  plt.xlabel("Beschriftung der x-Achse")
9  plt.ylabel("Beschriftung der y-Achse")
10 plt.title("Beispielgraph")
11 plt.legend()
12
13 plt.show()
```

Dies ist der vom obigen Codeabschnitt generierte Graph.



[24] https://matplotlib.org/3.1.1/api/markers_api.html

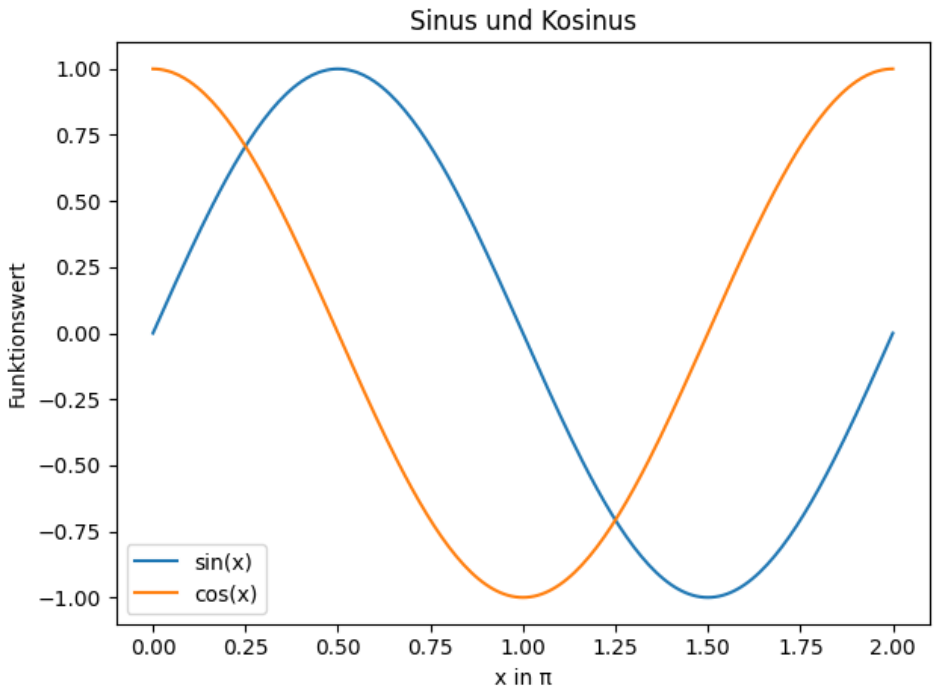
Die generierten Graphen können gespeichert werden, indem die **`savefig()`** Funktion nach allen Einstellungen, aber vor Aufruf der **`show()`** Funktion aufgerufen wird. Dabei ist das erste Argument der Dateiname. Über das **`format`** Keyword Argument kann ein Format angegeben werden, beispielsweise **`png`** oder **`pdf`**. Der Vorteil von PDF Dateien ist, dass es sich um Vektorgrafiken handelt. Der Graph kann also anschließend beliebig groß dargestellt werden, ohne unscharf zu werden.

```
1 plt.savefig("dateiname.pdf", format='pdf')
```

Nachfolgend werden wir zwei CSV Dateien einlesen, und deren Inhalt in Form eines Graphen ausgeben. Die beiden Eingabedateien **`sin.csv`** und **`cos.csv`** (stehen im ISIS Kurs zum Download bereit) enthalten in der ersten Spalte die x-Werte in π, sodass $1 = \pi$, $2 = 2\pi$ etc. Die zweite Spalte gibt die Funktionswerte **`sin(x)`** bzw. **`cos(x)`** an. Die ersten Zeilen der **`sin.csv`** sehen wie folgt aus:

```
1 0;0
2 0.01;0.031410759
3 0.02;0.06279052
4 0.03;0.094108313
5 0.04;0.125333234
6 0.05;0.156434465
```

Bringen wir unser bisher erlerntes Wissen zusammen, so können wir die Daten der CSV Dateien in diesem Graphen bündeln:



Der Graph wurde vom folgenden Codeabschnitt generiert:

```
import matplotlib.pyplot as plt
import csv

sin_x = []
sin_y = []
with open("sin.csv", "r", encoding="utf-8") as f:
    reader = csv.reader(f, delimiter=";")
    for row in reader:
        sin_x.append(float(row[0]))
        sin_y.append(float(row[1]))

cos_x = []
cos_y = []
with open("cos.csv", "r", encoding="utf-8") as f:
    reader = csv.reader(f, delimiter=";")
    for row in reader:
        cos_x.append(float(row[0]))
        cos_y.append(float(row[1]))

plt.plot(sin_x, sin_y, label="sin(x)")
plt.plot(cos_x, cos_y, label="cos(x)")

plt.xlabel("x in π")
plt.ylabel("Funktionswert")
plt.title("Sinus und Kosinus")
plt.legend()

plt.savefig("sin_cos.png", format="png")
plt.show()
```

7.4 Ausnahmebehandlung

Es kann während der Ausführung eures Skripts zu unvorhergesehenen Fehlern kommen. Zum Beispiel, wenn ihr auf den dritten Eintrag einer dynamisch generierten Liste zugreifen wollt, diese aber nur zwei Elemente hat. Kommt es zu einem solchen Fall, stürzt das Skript ab. Mittels Ausnahmebehandlung („Exception Handling") lässt sich definieren, wie sich das Programm in einem solchen Fall verhalten soll. Dadurch ist es möglich, das Skript vor einem Absturz zu bewahren.

Im folgenden Beispiel wird auf den dritten Eintrag (Index: 2) einer Liste mit zwei Elementen zugegriffen.

```
list = [0, 1]
print(list[2])
print("Fertig")
```

Dabei wird folgende Exception („Ausnahme”) ausgegeben:

```
IndexError: list index out of range
```

Uns wird dadurch mitgeteilt, dass der von uns gewählte Index in der Liste nicht existiert. Das Wort „Fertig” wird nicht ausgegeben, da es zuvor zur Exception kam. Wir werden das Skript nun so umstellen, dass die Exception behandelt, also der Fehler abgefangen wird. Dafür betten wir den Codeabschnitt in einen sogenannten **`try`** Block ein. Es wird also, wie der Name verrät, versucht, den enthaltenen Codeabschnitt auszuführen. Zusätzlich folgt nach dem **`try`** Block ein **`except`** Block. Dieser wird aufgerufen, wenn im **`try`** Block eine Exception auftritt („geworfen wird”).

```
list = [0, 1]

try:
    print(list[2])
except:
    print("Der Index existiert nicht.")
print("Fertig")
```

Im **`try`** Block wird weiterhin ein IndexError geworfen. Deshalb wird nun der Codeabschnitt im **`except`** Block ausgeführt. Anschließend wird der Code unter dem **`except`** Block ganz normal ausgeführt. Die Ausgabe des gesamten Skripts ist damit:

```
Der Index existiert nicht.
Fertig
```

Aktuell wird jegliche Art von Exception im **`except`** Block behandelt. Man kann, und sollte, zusätzlich definieren, welche Art von Exception man behandeln will. In diesem Fall wollen wir nur Exceptions vom Typ **`IndexError`** behandeln. Dies können wir hinter dem **`except`** Keyword festlegen.

```
list = [0, 1]

try:
    print(list[2])
except IndexError:
    print("Der Index existiert nicht.")
print("Fertig")
```

Man kann beliebig viele **`except`** Blöcke definieren. Beim Zugriff auf Werte in Dictionaries kann zum Beispiel ein **`KeyError`** auftreten.

Im folgenden Beispiel wird eine Liste und ein Dictionary definiert. Anschließend wird auf verschiedene Werte der Datenstrukturen zugegriffen. Es gibt einen **`except`** Block für den Fall, dass ein Index in der Liste nicht existiert, und einen für den Fall, dass ein Key im Dictionary nicht existiert.

```
1  list = ["list_value_1", "list_value_2"]
2  dict = {
3      "key1": "dict_value_1",
4      "key3": "dict_value_3",
5  }
6
7  try:
8      print(list[0])
9      print(list[1])
10     print(dict["key1"])
11     print(dict["key2"])
12     print(dict["key3"])
13 except IndexError:
14     print("Der Index existiert nicht.")
15 except KeyError:
16     print("Der Key existiert nicht.")
17 print("Fertig")
```

Schaut man sich die untenstehende Ausgabe des Skripts an, so fällt auf, dass es in Zeile 11 zu einem KeyError gekommen sein muss, da die entsprechende Meldung ausgegeben wurde. Dies ist darauf zurückzuführen, dass der Key „key2" im Dictionary nicht definiert ist. Weiterhin fällt auf, dass der Wert zum Key „key3" ebenfalls nicht ausgegeben wurde. Dies liegt daran, dass die Ausführung des **`try`** Blocks mit einem Sprung in einen **`except`** Block beendet wird. Anschließend wird der Code nach den **`except`** Blöcken ausgeführt.

```
1  list_value_1
2  list_value_2
3  dict_value_1
4  Der Key existiert nicht.
5  Fertig
```

In den bisherigen Beispielen sind die Exceptions vorhersehbar, da der Code statisch, ohne Eingaben von außen, definiert ist. Hingegen bei Fällen, wo eine Eingabe verlangt wird oder Daten aus verschiedenen Quellen geladen werden, ist die Ausnahmebehandlung deutlich sinnvoller.

In den **`except`** Blöcken kann beliebiger Code ausgeführt werden. Wenn möglich, soll der Fehler nicht nur über die Konsole ausgegeben werden, sondern auch behoben werden. Beispielsweise kann bei einer fehlerhaften Eingabe um eine korrigierte Eingabe gebeten werden. Das folgende Skript fragt so lange nach einem Dictionary Key, bis ein tatsächlich vorhandener Key eingegeben wird.

```
dict = {
    "key1": "dict_value_1",
    "key3": "dict_value_3",
}

while True:
    key = input("Eingabe des Keys: ")

    try:
        print(dict[key])
        break
    except KeyError:
        print("Der Key existiert nicht. Bitte einen anderen Key
        ↪  eingeben.")

print("Fertig")
```

Es folgt ein Beispieldurchlauf des Skripts:

```
Eingabe des Keys: key2
Der Key existiert nicht. Bitte einen anderen Key eingeben.
Eingabe des Keys: key3
dict_value_3
Fertig
```

8 Einblick in objektorientierte Programmierung

Auch wenn wir die objektorientierte Programmierung (OOP) in den bisherigen Kapiteln vermieden haben, so war sie dennoch in unseren Übungen und Beispielen meistens präsent. Wir haben Objekte und Methoden von Klassen benutzt, ohne eigentlich von deren Existenz zu wissen.

Oder um es noch direkter zu formulieren: Python ist durch und durch eine objektorientierte Sprache. In Python sind alles Objekte: Integer, Strings, Boolean, Module, Funktionen, usw. Daher soll das letzte Kapitel dieses Skripts eine Einführung in die objektorientierte Programmierung sein.

8.1 Was ist OOP?

Objektorientierte Programmierung ist die Bezeichnung eines Programmierparadigmas. Die Grundidee dahinter ist, Programme zu schreiben, die aus miteinander interagierenden Objekten bestehen. Dabei werden Daten und zugehörige Funktionen gebündelt. Um konkrete Daten abzuspeichern bzw. individuelle Objekte zu erstellen, werden im Vorhinein Schemata – sogenannte Klassen – entworfen. Nach diesen „Bauanleitungen“ können dann beliebig viele Instanzen (konkrete Objekte) erzeugt werden.

Die „Bauanleitungen“, nach denen die **Objekte** erstellt werden, werden also **Klassen** genannt. Die Daten der konkreten Objekte werden in Variablen gespeichert, die **Attribute** genannt werden. Funktionen realisieren den zugehörigen Code zu den Daten und werden als **Methoden** bezeichnet.

8.2 Was sind die Vorteile von OOP?

Ein typisches Beispiel, das bei dieser Thematik gebracht wird, ist ein Bankkonto. Vereinfacht gesagt, gehören zu jedem Konto folgende Informationen: Kontoinhaber*in, Kontonummer und Kontostand. Außerdem gibt es bestimmte Handlungen, die man mit einem Konto durchführen können muss. Man kann ein neues Konto eröffnen, Geld muss ein- bzw. ausgezahlt werden können und die Kontoinformationen müssen abrufbar sein.

Mit dem bisherigen Wissen würde man womöglich ein verschachteltes Dictionary namens „accounts“ erstellen, das für jedes Konto einen Eintrag besitzt. Daneben gäbe es verschiedene Funktionen, die die erwünschten Fähigkeiten ermöglichen würden. Das folgende Beispiel zeigt eine mögliche Umsetzung in diesem Stil. In Zeile 4 sieht man die Funktion `create_account(name, forename)`, mit der ein neues Bankkonto erstellt werden kann. Aus den angegebenen Personendaten (hier nur der Name) wird über eine hash-Funktion eine Kontonummer generiert, die als Schlüssel für das Konto dient. Der Kontostand wird automatisch auf null gesetzt.

*Bei den print-Ausgaben im folgenden Beispiel wurden zum Teil **f-Strings** verwendet, wodurch eine kompaktere und übersichtlichere Schreibweise ermöglicht wird. Das „f“ vor den Anführungsstrichen (z.B. in Zeile 20) steht für **format** und führt dazu, dass Variablen direkt in den String eingebaut werden können. Hierzu setzt man die entsprechende Variable in geschweifte Klammern und kann beispielsweise bei float-Zahlen noch mit einem Doppelpunkt die Anzahl an Nachkommastellen angeben.*

```
accounts = {}    # Dictionary mit allen erstellten Konten

# Ein neues Konto anlegen, sofern dieses noch nicht vorhanden ist
def create_account(name, forename):
    ba_number = hash(name + forename) % 10**10
    if ba_number in accounts:
        print("Konto bereits vorhanden!")
        return False
    balance = 0.0
    accounts[ba_number] = {'name': name,
                           'forename': forename,
                           'balance': balance}
    return ba_number

# Geldbetrag abheben, sofern das Konto gedeckt ist
def withdraw_money(ba_number, amount):
    if ba_number in accounts:
        if accounts[ba_number]['balance'] >= amount and amount >= 0:
            accounts[ba_number]['balance'] -= amount
            print(f"{amount:.2f} € wurden vom Konto abgebucht.")
            return True
        else:
            print("Nicht genug Geld auf Konto!")
            return False
    else:
        print("Konto nicht vorhanden!")
        return False

# Geld auf Konto einzahlen
def deposit_money(ba_number, amount):
    if ba_number in accounts:
        accounts[ba_number]['balance'] += amount
        print(f"{amount:.2f} € wurden dem Konto gutgeschrieben.")
        return True
    else:
        print("Konto nicht vorhanden!")
        return False

# Übersichtliche Ausgabe der Kontoinformationen
def get_accinfo(ba_number):
    if ba_number in accounts:
        print(f"Das Konto gehört: {accounts[ba_number]['forename']}
        ↪  {accounts[ba_number]['name']} (Kontonummer: {ba_number})")
        print(f"Kontostand: {accounts[ba_number]['balance']:.2f} €")
    else: print("Konto existiert nicht.")
```

Schaut man sich dieses Beispiel an, so fällt auf, dass die Funktionen über den Code nur spärlich mit der Datenstruktur „accounts“ verbunden sind. Der Zugriff auf das Dictionary wird über die Funktionen zwar erleichtert, man ist aber nicht gezwungen, diese zu verwenden. Stattdessen könnte man einzelne Konten auch selber erstellen bzw. irgendwelche sinnfreien Einträge erzeugen, die das System nicht verarbeiten könnte:

```
accounts = {}    # Dictionary mit allen erstellten Konten

accounts["fake-konto"] = {'balance': 9999999999,
                          'name': 42}
```

Aus diesem Grund ist das Prinzip der **Datenkapselung** sehr wichtig. Unter Datenkapselung versteht man den **Schutz von Daten bzw. Attributen vor dem unmittelbaren Zugriff**. Anstatt wie bei einem Dictionary einen freien Zugriff auf die Daten zu ermöglichen, erfolgt dieser nur über entsprechende Methoden, die man auch als **Zugriffsmethoden** bezeichnet.

Generell werden zu einem bestimmten Attribut zwei Zugriffsmethoden geschrieben: Die eine liefert einem den Wert des Attributes – häufig als Getter (engl. *„to get“*) oder als Abfragemethode bezeichnet – und mit der anderen lässt sich der Wert eines Attributes verändern – häufig als Setter (engl. *„to set“*) oder als Änderungsmethode benannt.

Wenn von „Schutz von Daten bzw. Attributen vor dem unmittelbaren Zugriff“ gesprochen wird, dann bedeutet das in der Objektorientierten Programmierung, dass die Daten außen nicht sichtbar sind. Man spricht in diesem Zusammen auch häufig vom **Geheimnisprinzip**. Interne Informationen und Implementierungsdetails werden nach außen nicht preisgegeben bzw. Programmierer*innen werden nicht mit unnötigen Implementierungsdetails belastet. Die außen unbedingt nötigen Daten werden nur über definierte Schnittstellen bzw. Methoden nach außen sichtbar und benutzbar gemacht. Methoden, die nur intern benötigt werden, können auch durch Zugriffe durch Nutzer*innen geschützt werden. Datenkapselung und Geheimnisprinzip können auch unter dem allgemeineren Begriff **Datenabstraktion** zusammengefasst werden.

8.3 Klassen erstellen und Objekte generieren

Daten und Verarbeitungsfunktionen werden also zu Klassen zusammenfasst. Dabei werden die Daten einer solchen Klasse **Attribute** und die Verarbeitungsfunktionen **Methoden** genannt. Schematisch lässt sich die Konto-Klasse wie in der Tabelle rechts darstellen.

Konto
Attribute
Name
Vorname
Kontonummer
Kontostand
Methoden
Konto erstellen
Geld abheben
Geld einzahlen
Kontoinfos abrufen

Aus dieser Klasse lassen sich nun konkrete Objekte (auch Instanzen genannt) erstellen (auch instanziieren genannt), deren Struktur von der Klasse definiert wird.

Doch wie wird eine Klasse in Python erstellt? Dies soll an dem Konto-Beispiel von vorhin gezeigt werden. Nun in einer in die OOP umformulierten Version. Auf den ersten Blick wird euch bestimmt vieles fremd vorkommen. Aber alle Teile werden im Anschluss ausführlich erklärt.

Versucht in einem ersten Schritt lediglich die grobe Syntax zu verstehen und folgende Schlüsselbegriffe aufzufinden: **class**, **self**, **type(self)**, **__init__**, **__del__**, **__str__**

```
class account:
    list_of_accounts = []

    def __init__(self, name, forename):
        self.__name = name
        self.__forename = forename
        self.__ba_number = hash(name + forename) % 10**10
        self.__balance = 0.0
        type(self).list_of_accounts.append(self.__ba_number)

    def __del__(self):
        type(self).list_of_accounts.remove(self.__ba_number)

    def __str__(self):
        return f"Konto: {self.__forename} {self.__name},
        ↪  {self.__ba_number} ({self.__balance:.2f} €)"

    def withdraw_money(self, amount):
        if self.__balance >= amount and amount > 0:
            self.__balance -= amount
            print(f"{amount:.2f} € wurden vom Konto abgebucht.")
            return True
        else:
            print("Nicht genug Geld auf Konto oder eingegebener Betrag
            ↪  <= 0")
            return False

    def deposit_money(self, amount):
        self.__balance += amount
        print(f"{amount:.2f} € wurden dem Konto gutgeschrieben.")
        return True
```

Der Code zu einer Klasse wird mit dem Schlüsselwort **class**, gefolgt von dem Klassennamen begonnen.

Um nun ein konkretes Konto anzulegen, muss ein Objekt der `account`-Klasse instanziiert werden:

```
# Objekt erstellen
acc_x = account("Julia", "Schmidt")
```

Um festzulegen, welche Parameter zum Erstellen eines Objekts angegeben werden müssen, gibt es **__init__**. `__init__` ist eine Methode, die unmittelbar und automatisch während der Erzeugung einer

Instanz aufgerufen wird. Der Name ist dabei von Python festgelegt. Sie kann an beliebiger Stelle in der Klassendefinition stehen, sollte aber nach Konvention immer die erste Methode direkt unter dem Klassenheader sein. In diesem Beispiel wird bei der `account`-Klasse zum Erstellen die Angabe eines Namens und eines Vornamens verlangt. Wird versucht, ein Objekt der Klasse mit einer anderen Anzahl an Parametern zu erstellen, wird ein **`TypeError`** ausgelöst.

In der `init`-Methode (Zeile 4) sind drei Parameter angegeben. Wieso werden trotzdem nur zwei verlangt?
Der erste Parameter **`self`** ist kein normaler Parameter, sondern eine Referenz auf das Objekt selber. Wird ein neues Objekt erstellt, muss diesem Objekt eine Variable zugewiesen werden (im Beispiel `acc_x`). Python holt sich den Variablennamen selbstständig. Anders gesagt: **`self`** bezieht sich immer auf das konkrete Objekt. Daher werden damit gebildete Attribute (Zeile 5 bis 9) auch als ***Instanzattribute*** bezeichnet.

Im Gegensatz dazu wurde `self` bei `list_of_accounts` bewusst weggelassen. Es handelt sich also nicht um ein Instanzattribut, sondern um ein ***Klassenattribut***, das sich alle Objekte der Klasse miteinander teilen. In diesem Beispiel soll `list_of_accounts` dabei helfen, nicht die Übersicht über die erstellten Konten zu verlieren. Wird ein neues Konto erstellt, wird die Kontonummer zur Liste hinzugefügt (Zeile 9). Wird hingegen ein Konto mittels **`del`** gelöscht, sorgt die Definition des **Destruktors** **`__del__`** (Zeile 11,12) dafür, dass die entsprechende Kontonummer aus der Liste entfernt wird.

Neben `__init__` und `__del__` gibt es noch weitere „*magische*“ Methoden mit doppelten Unterstrichen. Die Namen sind dabei alle von Python festgelegt und können nicht frei gewählt werden! Auf welche an dieser Stelle noch kurz eingegangen werden soll, ist `__str__` (Zeile 14 und 15). Besitzt eine Klasse eine `__str__`-Funktion, dann wird diese genutzt, wenn auf ein Objekt dieser Klasse `str()` angewendet wird, oder wenn ein Objekt dieser Klasse in einer print-Funktion verwendet wird.

```
1 acc_x = account("Julia", "Schmidt")
2
3 print(acc_x)     # Konto: Schmidt Julia, 1741936205 (0.00 €)
```

8.3.1 Zugriffsbeschränkung: Private Attribute

Die **doppelten Unterstriche** zu Beginn der `init`-Methode und auch der verschiedenen Attribute (z.B. `__name`, `__forename`, etc. in Zeile 5 ff.) machen diese **privat**. Das bedeutet, dass die jeweiligen Methoden/Attribute nur intern sichtbar sind.
Würde man von außen (z.B. über `mein_objekt.private_methode()` oder auch über einen Aufruf in der Konsole) versuchen, auf sie zuzugreifen, erhält man einen **`AttributeError`**. Die privaten Methoden/Attribute sind also nach außen hin nicht vorhanden und damit vor unerwünschten Veränderungen geschützt.

```
class example:

    def __priv_method(self):
        print("Private Methode wurde aufgerufen")

    def pub_method(self):
        self.__priv_method()

a = example()
a.pub_method()          # Private Methode wurde aufgerufen
a.__priv_method()       # AttributeError: 'example' object has no
 ↪  attribute '__priv_method'
```

8.4 Vererbung

Neben der strukturellen Verschmelzung von Daten und den darauf arbeitenden Methoden zu einer Einheit zielt das Konzept der Objektorientierung darauf ab, die Wiederverwendbarkeit von Programm-Code zu verbessern.

Konkret bedeutet dies, dass man von bereits bestehenden Klassen (*parent classes*) neue untergeordnete Klassen (*child classes*) ableitet, um diese um zusätzliche Funktionalität zu erweitern. Dabei übernimmt die untergeordnete Klasse alle Fähigkeiten der übergeordneten Klasse, sodass sie zunächst eine Kopie dieser Klasse ist. Man sagt, die Basisklasse vererbt ihre Fähigkeiten an untergeordnete Klassen. Nach diesem Vererbungsschritt kann die abgeleitete Klasse um weitere Methoden/Attribute erweitert werden.

```
class A:
    def __init__(self):
        self.X = 1337
        print("Konstruktor von A")
    def m(self):
        print("Methode m von A. self.X =", self.X)

class B(A):          # B erbt von A
    def n(self):
        print("Methode n von B")


b = B()      # Konstruktor von A
b.n()        # Methode n von B
b.m()        # Methode m von A. self.X = 1337
```

In diesem Beispiel gibt es neben der Klasse A noch die untergeordnete Klasse B. Der in Klammern angegebene Klassennamen gibt die übergeordnete Klasse an.[25]

Am Ende des kleinen Programms werden eine Instanz der Klasse B erzeugt und ihre Methoden n und m aufgerufen. Die Ausgabe zeigt, dass B sowohl den Konstruktor als auch die Methode m von der Klasse A geerbt hat. Auch das Attribut X wurde ordnungsgemäß angelegt.

8.4.1 Überschreiben von Methoden

In der Praxis ist es häufig so, dass eine abgeleitete Klasse einen anderen Konstruktor als ihre übergeordnete Basisklasse benötigt, um eigene Initialisierungen vorzunehmen. Daher wird Klasse B um einen eigenen Konstruktor erweitert, der ein Attribut Y anlegt und auch eine Ausgabe erzeugt. Zusätzlich wird die Methode n erweitert, dass sie den Wert des Attributs Y ausgibt.

```
1  class A:
2      def __init__(self):
3          self.X = 1337
4          print("Konstruktor von A")
5      def m(self):
6          print("Methode m von A. self.X =", self.X)
7
8  class B(A):
9      def __init__(self):
10         self.Y = 10000
11         print("Konstruktor von B")
12     def n(self):
13         print("Methode n von B. self.Y =", self.Y)
14
15
16 b = B()       # Konstruktor von B
17 b.n()        # Methode n von B. self.Y = 10000
18 b.m()        # !! AttributeError: 'B' object has no attribute 'X'
```

Die Klasse B hat die `__init__`-Methode — also den Konstruktor -– zunächst von der Klasse A geerbt, sie aber dann mit ihrem eigenen Konstruktor überschrieben. Infolgedessen wird beim Erzeugen einer Instanz der Klasse B nur noch der neue von B definierte Konstruktor aufgerufen, während der Konstruktor von A ignoriert wird.

Generell spricht man vom Überschreiben einer Methode, wenn eine Klasse eine Methode erneut implementiert, die sie bereits von ihrer Basisklasse geerbt hat.

Oftmals ist es allerdings einfacher, wenn der Konstruktor der Basisklasse nicht ersetzt (überschrieben) sondern ergänzt wird. Mittels **`super()`** ist es möglich, überschriebene Methoden der Basisklasse explizit zu rufen.

[25] Eine Klasse kann auch von mehreren anderen Klassen erben. z.B. `class C(A, B)`

Im folgenden Beispiel wird beim Erstellen eines Objekts der Klasse B auch der Konstruktor von Klasse A explizit aufgerufen. Dadurch erzeugt der Aufruf `b.m()` auch keine Fehlermeldung.

```
class A:
    def __init__(self):
        self.X = 1337
        print("Konstruktor von A")
    def m(self):
        print("Methode m von A. self.X =", self.X)

class B(A):
    def __init__(self):
        super().__init__()
        self.Y = 10000
        print("Konstruktor von B")
    def n(self):
        print("Methode n von B. self.Y =", self.Y)


b = B()      # 'Konstruktor von A', 'Konstruktor von B'
b.n()        # Methode n von B. self.Y = 10000
b.m()        # Methode m von A. self.X = 1337
```

Dieses Überschreiben ist selbstverständlich nicht auf den Konstruktor beschränkt. Das heißt, es kann jede beliebige Methode der Basisklasse – wie der Konstruktor im obigen Beispiel – explizit gerufen werden.